

**MINISTRY OF EDUCATION
AND TRAINING**

**MINISTRY OF NATIONAL
DEFENCE**

MILITARY TECHNICAL ACADEMY



FULL NAME: DAM VU DUC ANH

COURSE: 57

TRAINING TYPE: MILITARY ENGINEER

GRADUATION THESIS

MAJOR: INFORMATION SECURITY

ID:

SUB-MAJOR: INFORMATION SECURITY ASSURANCE

**RESEARCH ON MULTI-EDGE COORDINATION
TECHNOLOGY APPLIED TO PROCESSING MULTIPLE
TRAFFIC CAMERAS**

2026

**MINISTRY OF EDUCATION
AND TRAINING**

**MINISTRY OF NATIONAL
DEFENCE**

MILITARY TECHNICAL ACADEMY



FULL NAME: DAM VU DUC ANH

COURSE: 57

TRAINING TYPE: MILITARY ENGINEER

GRADUATION THESIS

MAJOR: INFORMATION SECURITY

ID:

SUB-MAJOR: INFORMATION SECURITY ASSURANCE

**RESEARCH ON MULTI-EDGE COORDINATION
TECHNOLOGY APPLIED TO PROCESSING MULTIPLE
TRAFFIC CAMERAS**

Supervisor: Lieutenant Colonel, PhD. Dang Le Dinh Trang

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên: Đàm Vũ Đức Anh

Lớp: Mạng máy tính và truyền thông dữ liệu Khóa: 57

Ngành: Mạng máy tính Chuyên ngành: Mạng máy tính và truyền
thông dữ liệu

1. Tên đề tài:

Nghiên cứu công nghệ Multi-Edge Coordination ứng dụng trong xử lý nhiều
camera giao thông.

2. Các số liệu ban đầu:

Tự tìm hiểu.

3. Nội dung bản thuyết minh:

Gồm 03 chương chính:

- Chương 1: Tổng quan đề tài và cơ sở lý thuyết;
- Chương 2: Phân tích và thiết kế giải pháp Multi-Edge Coordination;
- Chương 3: Xây dựng, phát triển và đánh giá hệ thống;

4. Số lượng, nội dung các bản vẽ và các sản phẩm cụ thể:

- Không có bản vẽ.

5. Cán bộ hướng dẫn:

Họ và tên: TS. Đặng Lê Đình Trang

Cấp bậc: Trung tá

Chức vụ: Phó chủ nhiệm Bộ môn

Đơn vị: Bộ môn An toàn thông tin - Công nghệ mạng - Viện Công nghệ
thông tin và Truyền thông - Học viện Kỹ thuật Quân sự

Nội dung hướng dẫn: Toàn bộ đồ án

Ngày giao: /7/2026

Ngày hoàn thành: /10/2026

Hà Nội, ngày tháng 10 năm 2026

Chủ nhiệm bộ môn

Cán bộ hướng dẫn

3// GV, TS. Cao Văn Lợi

2// GV, TS. Đặng Lê Đình Trang

Học viên thực hiện

Đã hoàn thành và nộp đồ án ngày tháng 10 năm 2026

Đàm Vũ Đức Anh

Abstract

In recent years, security researchers have encountered a significant evolution in malware development. Not only have new techniques been introduced, but malware development has also become more systematic, particularly through the use of packers. A large portion of modern malware is obfuscated using packers to evade detection by antivirus software. For the x86 architecture, there are over 50 popular packers readily available online, many of which encrypt the payload, which is then decrypted at runtime by unpacking stubs. Therefore, identifying the packers used, detecting the Original Entry Point (OEP), and dumping the original payloads are crucial steps in reversing binaries and analyzing malware.

This thesis explores the use of dynamic symbolic execution in binary analysis and delves into the structure of the Binary Emulation for Pushdown Model (BE-PUM), a project aimed at analyzing and detecting packers in binary files. The primary focus is on generating a Control Flow Graph (CFG) for packed files. Additionally, the study applies graph similarity techniques for OEP detection, develops a Command-line Interface (CLI) application, and automates the process of dumping original payloads and rebuilding Import Address Tables (IATs) for packed Windows x86 Portable Executable (PE) files.

Acknowledgements

Upon completing this thesis, I would like to express my profound gratitude to those who have supported me throughout this journey.

Firstly, I am deeply thankful to my beloved family for their constant support and encouragement, without which this study would not have been a possible mission.

I extend my sincere appreciation to my supervisor, Lecturer PhD. Phan Viet Anh, for his exemplary guidance and continuous encouragement throughout my research. His insight and expertise have been invaluable in helping me complete this work. I am also grateful to all the teachers at the Institute of Information and Multimedia Technology for equipping me with the necessary knowledge over the past five years. Additionally, I would like to give a big shout-out to Professor Ogawa and all the members of the Ogawa Laboratory, especially Pham Thanh Hung-san, for their kindness, support during my time as a short-term intern from Vietnam and their constant academic assistance during the time I conducted this research.

Lastly, I express my gratitude to the officers at all levels within my unit and all my peers at Military Technical Academy - MTA, especially MTA Security Club - MSEC members. Their camaraderie, thoughtful advice, and unwavering support have been a source of strength and motivation of mine. Thanks for standing by me through every challenge and triumph.

This thesis is not just a culmination of my efforts but a testament to the incredible people who have walked this path with me. To all of you, I offer my deepest thanks and heartfelt appreciation.

Abbreviations

No.	Abbreviation	Full Form
1	AI	Artificial Intelligence - Trí tuệ nhân tạo
2	API	Application Programming Interface - Giao diện lập trình ứng dụng
3	CPU	Central Processing Unit - Đơn vị xử lý trung tâm
4	CUDA	Compute Unified Device Architecture - Kiến trúc tính toán GPU
5	FPS	Frames Per Second - Số khung hình trên giây
6	GPU	Graphics Processing Unit - Đơn vị xử lý đồ họa
7	IP	Internet Protocol - Giao thức Internet
8	LPD	License Plate Detection - Phát hiện biển số
9	LPR	License Plate Recognition - Nhận dạng biển số
10	MQTT	Message Queuing Telemetry Transport - Giao thức nhắn tin IoT
11	P2P	Peer-to-Peer - Ngang hàng
12	ROI	Region of Interest - Vùng quan tâm
13	RTSP	Real-Time Streaming Protocol - Giao thức truyền phát thời gian thực
14	SGIE	Secondary GIE - Bộ suy luận thứ cấp trong DeepStream
15	YOLO	You Only Look Once - Mô hình phát hiện đối tượng thời gian thực

TABLE OF CONTENTS

INTRODUCTION.....	1
Motivations and Problems.....	1
Contribution	1
Outline	1
Chapter 1. BINARY ANALYSIS ON X86, X64 ARCHITECTURE	3
1.1. Binary Analysis Problems	3
1.1.1. Types of Binary Analysis	3
1.1.2. Applications of Binary Analysis	4
1.2. x86, x64 Architecture	6
1.2.1. x86 Architecture	6
1.2.2. x64 Architecture	14
1.3. PE File Format	17
1.4. Structure of Packed Code	23
1.5. Binary Analysis Difficulties	28
1.5.1. How Obfuscation Hinders Malware Analysis	28
1.5.2. Deep Inspection of API Obfuscation Techniques	30
1.6. Malware Analysis: Classical Methods and New Trends of Using Dynamic Symbolic Execution	32
1.7. Conclusion	36
Chapter 2. DYNAMIC SYMBOLIC EXECUTION	37
2.1. Symbolic Execution and Dynamic Symbolic Execution in Binary Analysis	37
2.2. Introduction of BE-PUM	41
2.3. Introduction of Miasm	42
2.4. Introduction of Triton	43
2.5. Conclusion	44
Chapter 3. ROLES OF CONTROL FLOW GRAPH AND ORIGINAL ENTRY POINT DETECTION	45
3.1. Roles of Control Flow Graphs	45

3.2. OEP Detection Method Based on Graph Similarity	48
3.2.1. OEP Detection.....	48
3.2.2. OEP Detection Based on Graph Similarity	50
3.2.3. Graph Similarity Using Weisfeiler-Lehman Kernel.....	52
3.2.4. Control Flow Graphs of Unpacking Stubs	54
3.3. Conclusion	55
Chapter 4. IMPLEMENTATION AND EXPERIMENTS	57
4.1. Implementation	57
4.1.1. Data Preparation Module	57
4.1.2. Template Setup Module.....	60
4.1.3. Packer Identification and OEP Detection Module.....	64
4.1.4. IAT Rebuild Module	65
4.1.5. Diagram of the Solution	69
4.2. Experiments	71
4.2.1. Packer Identification and CFG Generation Using BE-PUM	71
4.2.2. Packer Recognition And OEP Detection Based On Graph Similarity	72
4.2.3. Extracting Original Payloads Manually	76
4.2.4. Extracting Original Payloads Using PE Dump and IAT Rebuild Module	80
4.3. Conclusion	84
CONCLUSION	86
Achievements.....	86
Limits.....	87
Future Works	88
BIBLIOGRAPHY	90

LIST OF FIGURES

Figure 1.1. 32-bit general-purpose registers.....	8
Figure 1.2. PE file format	19
Figure 1.3. Structure of a PE file	20
Figure 1.4. Packing and unpacking process	24
Figure 1.5. An example of UPX packed code.....	24
Figure 1.6. Illustration of different API obfuscation schemes	31
Figure 2.1. Symbolic execution example	38
Figure 3.1. Illustration of Weisfeiler-Lehman Kernel algorithm.....	53
Figure 4.1. Dataset for training and testing packer identification and original entry point detection.....	57
Figure 4.2. Clustering function.....	61
Figure 4.3. Calculate feature vectors' frequencies	62
Figure 4.4. Feature vectors (label + frequency) of each packer.....	62
Figure 4.5. Function to form templates for each packer	63
Figure 4.6. Function to save end sequences of unpacking stubs.....	63
Figure 4.7. End sequences of unpacking stubs.....	64
Figure 4.8. End of unpacking prediction.....	64
Figure 4.9. Final decision for packer identification and OEP detection	65
Figure 4.10. Section Hdrs information.....	66
Figure 4.11. Diagram of the solution supporting reversing packed files	69
Figure 4.12. Diagram of the solution supporting reversing packed files (zoomed- in image)	70
Figure 4.13. Execution of BE-PUM in test 1	71
Figure 4.14. Log file of BE-PUM test 1	71
Figure 4.15. Execution of BE-PUM in test 2	72
Figure 4.16. Evaluation process.....	72
Figure 4.17. Results of packer and OEP detection of BE-PUM and graph-based method	73
Figure 4.18. Several versions of Winupack handled by DIE.....	73
Figure 4.19. Two versions of WINUPACK v.039f / Alt stub detected by DIE .	74
Figure 4.20. Two special templates of WinUpack unpacking stubs	74
Figure 4.21. Information of the input packed file	75
Figure 4.22. Demo with MPRESS_pestudio.exe file	75
Figure 4.23. Result of graph-based method and BE-PUM	76
Figure 4.24. Compare with the original file	76

Figure 4.25. Load file into x32dbg	77
Figure 4.26. Locate to the OEP	77
Figure 4.27. Get imports successfully.....	78
Figure 4.28. Dump, fix dump and rebuild IAT with Scylla.....	78
Figure 4.29. Comparison between two IATs	79
Figure 4.30. DLL characters shown in CFF Explorer	80
Figure 4.31. Result of dumping an fixing IATs module.....	81
Figure 4.32. Function to process dumping and rebuilding IATs	81
Figure 4.33. Successfully dumping payload	82
Figure 4.34. Initial IAT	82
Figure 4.35. Reconstructed IAT	83
Figure 4.36. Functions called from kernel32.dll to ntdll.dll	83
Figure 4.37. Graph comparison	84

LIST OF TABLES

Table 1.1. Names for different portions of 64-bit registers	15
---	----

INTRODUCTION

Motivations and Problems

Cyberattacks are on the rise, increasingly targeting governments, the military, and both the public and private sectors. The motivations behind these attacks are varied, ranging from espionage aimed at exfiltrating valuable information, to financial gain through ransomware, to sabotage intended to damage assets and reputations.

In particular, x86 malware continues to present significant threats, as the x86 architecture remains widely used across a broad spectrum of devices, including personal computers, servers, and embedded systems. This widespread use offers malware authors a large and accessible attack surface.

Given the sophistication of modern malware, the key challenge is determining how to analyze these threats effectively and efficiently.

Contribution

The main objective of this work is to study Dynamic Symbolic Execution (DSE) on binary analysis, understand how a DSE-based tool (BE-PUM, Triton, Miasm,...) works. Take advantages of BE-PUM's result (log file and CFG) to identify packers (12 packers in total) and detect Original Entry Point of a packed Windows x86 PE files adopting the graph similarity. At the moment, we have rebuilt the templates for each packer to increase percent of precise detection; built a CLI app to detect packer and OEP; developed a tool to automate the process of rebuilding IAT. Besides, we have added several different API obfuscation techniques that BE-PUM has not mentioned. They all need considering when we would like to successfully dump the original executable payload.

Outline

My thesis flows according to the following structure. Chapter 1 describes x86 architecture and several problems with reversing binaries including packing, API obfuscating and so on.

Chapter 2 studies about dynamic symbolic execution and work of BE-PUM, a DSE-based tool to generate CFGs of x86 PE files.

Chapter 3 investigates the method of OEP detection by analyzing the similarity between two CFGs.

Chapter 4 details the implementation and experiments of these aforementioned methods.

Finally the last chapter wraps up the achievements and future development of the research to be done.

Chapter 1. BINARY ANALYSIS ON X86, X64 ARCHITECTURE

1.1. Binary Analysis Problems

Binary analysis is the process of examining and understanding compiled executable code in its binary format, without the need for the original source code. This type of analysis is crucial in many areas of software development, security research, and reverse engineering. Unlike high-level analysis, which deals with the original source code, binary analysis focuses on the machine-level instructions that are executed directly by the computer's hardware. This makes it an invaluable tool in scenarios where the source code is unavailable, such as in closed-source software, malware analysis, or legacy systems.

1.1.1. Types of Binary Analysis

There are two primary methods used in binary analysis:

Static analysis: In this approach, the binary code is analyzed without executing it. Techniques like disassembly and decompilation are employed to reconstruct the program's control flow, identify data structures, and uncover potential vulnerabilities or malicious code. Static analysis is useful because it is faster and does not require the code to run, making it safer when dealing with potentially harmful binaries.

Static analysis may struggle with obfuscated or packed binaries, where the code is hidden or altered to prevent easy inspection.

Dynamic analysis: Dynamic analysis entails running the binary in a controlled environment, such as a virtual machine or sandbox, to monitor its behavior during execution. This approach enables analysts to observe how the binary interacts with the operating system, network, and other resources. It is especially useful for detecting runtime behaviors, including system calls, memory usage, and network activity.

Some malware or sophisticated binaries can detect when they are being run in a virtualized environment and modify their behavior accordingly, making dynamic analysis more challenging.

1.1.2. Applications of Binary Analysis

Binary analysis plays a pivotal role across various domains in computer science and cybersecurity, offering insights into software behavior without requiring access to its source code. By analyzing compiled binaries at the machine level, it enables the identification of vulnerabilities, the reverse engineering of software, and the detection of malicious behavior. Whether in software development, digital forensics, or cybersecurity, binary analysis has become an indispensable tool for understanding the inner workings of complex systems. Below are some key areas where binary analysis is applied, showcasing its versatility and importance in modern technological landscapes.

Malware analysis: One of the most critical applications of binary analysis is in cybersecurity, particularly for analyzing and understanding malware. Malware often comes in binary form, and understanding its structure and behavior is essential for creating detection tools, preventing infections, and mitigating its impact. Both static and dynamic analysis techniques are used to reverse-engineer malware, detect vulnerabilities, and uncover the intent and capabilities of the malicious code.

Reverse engineering: Binary analysis is heavily used in reverse engineering to understand how a program works when the source code is not available. This is common in software security audits, legacy software maintenance, or understanding third-party software. Reverse engineering through binary analysis also helps to identify proprietary algorithms, bypass software protection mechanisms, and detect vulnerabilities in software.

Vulnerability detection: Binary analysis tools are commonly used in vulnerability detection to find security weaknesses in software. By examining the binary, analysts can uncover flaws such as buffer overflows, format string vulnerabilities, and memory corruption issues. This type of analysis is crucial for ensuring the security of software, especially in critical systems where vulnerabilities could lead to severe consequences.

Performance optimization: Binary analysis can also be applied to optimize the performance of software. By analyzing how a binary interacts with the hardware, it is possible to identify bottlenecks, inefficient memory usage, or suboptimal code paths that can be improved to enhance overall performance. This is particularly important in embedded systems or high-performance computing, where efficiency is key.

Software debugging: In cases where debugging information is not available or the source code is missing, binary analysis is used to perform software debugging. This helps in identifying and fixing issues at the machine-code level, especially in situations where traditional debugging tools cannot be applied.

Digital forensics: Binary analysis plays a crucial role in digital forensics, where investigators analyze binaries found in compromised systems to understand the origin and impact of cyberattacks. By dissecting these binaries, forensics experts can trace back actions performed by the malicious code and determine the full scope of an attack.

The importance of binary analysis becomes even more pronounced in malware analysis, where understanding the behavior and intent of malicious software is critical for developing effective defense mechanisms. Malware is often distributed in binary form and frequently employs techniques such as obfuscation, packing, and encryption to evade detection. In this context, binary analysis is indispensable for uncovering how malware operates, identifying its command and control infrastructure, and discovering any hidden payloads or malicious functionality of its.

The objective of this thesis is to enhance the field of malware analysis by focusing on techniques that can overcome these evasion strategies, specifically through methods such as dynamic symbolic execution and advanced unpacking techniques. By applying these approaches, the research aims to provide deeper insights into modern malware, ultimately contributing to more effective detection and mitigation solutions in cybersecurity.

1.2. x86, x64 Architecture

To fully grasp the complexities of modern binary analysis, it is essential to understand the underlying hardware architectures that these binaries are designed to run on. The x86 and x64 architectures are two of the most commonly used instruction sets in today's computing environments. These architectures define how machine code interacts with the hardware, dictating the structure of executable files, memory management, and instruction sets.

Analyzing binaries compiled for these architectures requires a solid understanding of their features and capabilities. In the following sections, we will explore the x86 and x64 architectures, starting with an in-depth look at the x86 family, which forms the foundation of many current systems and has evolved into the x64 architecture. This understanding is critical for performing effective reverse engineering and malware analysis on modern software.

1.2.1. x86 Architecture

The x86 architecture refers to a family of backward-compatible instruction set architectures. The name "x86" originated from the series of Intel processors that followed the 8086, such as the 80186, 80286, 80386, and 80486, which all ended in "86." Today, x86 typically refers to binary compatibility with the 32-bit x86 instruction set. It follows the Complex Instruction Set Computing (CISC) design, where a single instruction can perform multiple low-level operations, such as loading from memory, performing an arithmetic operation, and storing back to memory. The x86 instruction set includes assembly instructions supported by x86-compatible processors, featuring variable-length binary instructions and flexible operand formats, ranging from none to up to three operands.

1.2.1.1. Components of x86 32-bit Processor

A typical x86 32-bit processor is designed with several key components that work together to perform various computing tasks. These components include general-purpose registers, segment registers, control registers, and other essential

units that enable efficient processing and management of instructions, memory, and input/output operations. Below is an overview of the major components:

General-purpose registers: The x86 32-bit processor contains eight general-purpose registers, each with a specific function, although they can be used interchangeably for different purposes in many cases. These registers handle arithmetic, memory addressing, and data movement within the processor:

- **Accumulator Register (AX):** Primarily used for arithmetic and logic operations, it plays a central role in most computational tasks.
- **Counter Register (CX):** Typically used in loop control, shift/rotate operations, and as a counter for repeated string operations.
- **Data Register (DX):** Involved in arithmetic operations and is often used for input/output (I/O) operations.
- **Base Register (BX):** Holds memory addresses for data pointers, particularly when working with data in the data segment.
- **Stack Pointer (SP):** Points to the current position at the top of the stack, which is used to store return addresses, local variables, and control data.
- **Base Pointer (BP):** Points to the base of the stack frame, used to reference function parameters and local variables within the current stack frame.
- **Source Index (SI):** Acts as a pointer to the source for string operations, particularly in memory-to-memory transfers.
- **Destination Index (DI):** Points to the destination for string operations, used in conjunction with the SI register for memory transfers.

General-Purpose Registers

Named storage locations inside the CPU, optimized for speed.

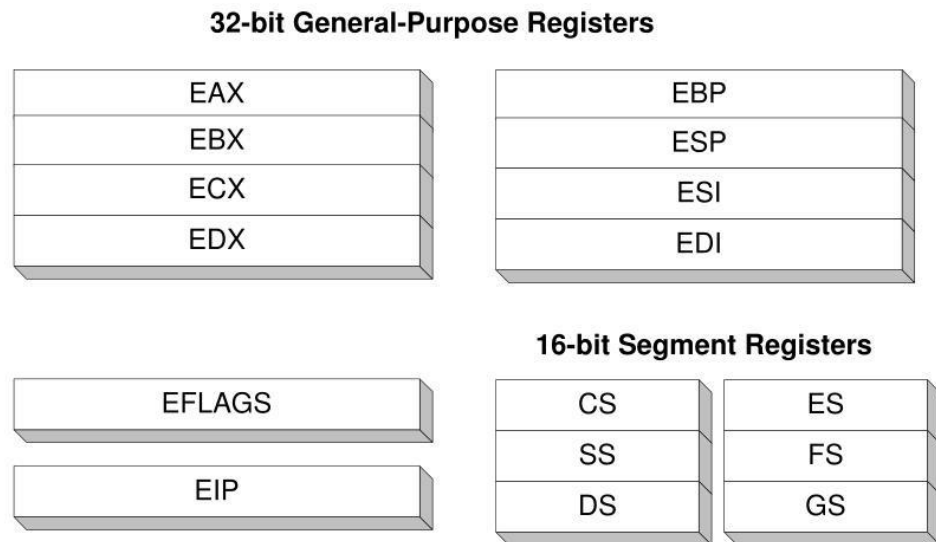


Figure 1.1. 32-bit general-purpose registers

General-purpose registers can be accessed in both 16-bit and 32-bit modes. In 16-bit mode, registers are referred to by their two-letter abbreviations, as listed earlier. In 32-bit mode, these abbreviations are prefixed with the letter "E" (for extended), such as "EAX" for the 32-bit accumulator register. Additionally, the first four registers (AX, CX, DX, and BX) can be accessed in two 8-bit parts by replacing the letter "X" with "H" for the higher 8 bits or "L" for the lower 8 bits. For example, "AH" refers to the higher 8 bits of AX, and "AL" refers to the lower 8 bits of the register.

Segment registers:

Segment registers store memory segment addresses, which are used to divide the computer's memory into manageable sections. These registers help in addressing specific locations in memory.

- **Code Segment (CS):** Points to the segment containing the current instruction being executed.
- **Data Segment (DS):** Points to the segment where data is stored.
- **Stack Segment (SS):** Points to the segment containing the stack, which is used for function calls, local variables, and managing return addresses.
- **Extra Segment (ES), FS, GS:** Additional segment registers used to handle memory more flexibly, particularly for data storage and manipulation.

Instruction Pointer (IP): The Instruction Pointer (also known as the Program Counter) keeps track of the address of the next instruction to be executed. It plays a crucial role in controlling the flow of execution within a program.

Flags register (EFLAGS): The EFLAGS register is a 32-bit register that stores a set of Boolean flags, each representing specific characteristics of operation results and the processor's current state. The key flags are as follows:

- **Carry Flag (CF):** Set when the last arithmetic operation results in a carry (for addition) or a borrow (for subtraction) that extends beyond the most significant bit (leftmost) of the register.
- **Parity Flag (PF):** Cleared if the number of 1-bits in the result of the last operation is even; otherwise, it is set.
- **Adjust Flag (AF):** Indicates a carry from the lower nibble (4 bits) during an arithmetic operation, particularly when deal with Binary-Coded Decimal (BCD) numbers.
- **Zero Flag (ZF):** Set when the result of an operation is zero; otherwise, it remains cleared.
- **Sign Flag (SF):** Indicates the sign of the result of an operation; it is set to 1 if the result is negative.
- **Trap Flag (TF):** Used to enable single-step debugging, allowing the processor to execute one instruction at a time.
- **Interrupt Flag (IF):** Set if interrupts are enabled; cleared otherwise.

- Direction Flag (DF): Controls the direction for string operations, determining whether the processing moves left or right through memory.
- Overflow Flag (OF): Set when the result of a signed arithmetic operation is too large to fit in the destination register.

These flags help the processor determine the outcome of instructions and manage program flow based on the results of executed operations.

1.2.1.2. Addressing Modes in x86 Architecture

One of the essential features of the x86 architecture is its flexible addressing modes, which allow various ways to reference memory and registers. The addressing modes in x86 architecture include:

Immediate addressing: In this mode, the operand is a constant value that is specified directly within the instruction. The value is not retrieved from memory or a register; instead, it is embedded in the instruction itself. For example, the instruction `MOV AX, 10` moves the constant value 10 directly into the AX register. Similarly, `ADD BX, 3` adds the value 3 directly to the contents of BX.

Register addressing: In this addressing mode, the operand is located in one of the processor's general-purpose registers. The instruction specifies which register holds the operand. For example, the instruction `MOV AX, CX` copies the value currently stored in the CX register into the AX register. Another example is `ADD DX, BX`, which adds the value in the BX register to the DX register.

Direct addressing: In direct addressing mode, the operand is stored in memory, and the instruction directly provides the memory address where the operand is located. For instance, the instruction `MOV AX, [2000h]` moves the value located at memory address 2000h into the AX register. Likewise, `ADD BX, [3000h]` adds the value at memory address 3000h to the BX register.

Indirect addressing: This mode uses registers as pointers to memory locations. Instead of directly specifying a memory address, the instruction uses the content of a register to point to the memory address where the operand is stored. For example, in `MOV AX, [BX]`, the value stored at the memory location

pointed to by the BX register is moved into the AX register. Similarly, ADD CX, [SI] adds the value at the memory location pointed to by the SI register to CX.

Indexed addressing: In indexed addressing, the memory address of the operand is calculated by adding a base address and an index. This mode is particularly useful for accessing arrays or data tables in memory, as the index can vary to access different elements. For example, the instruction MOV AX, [BX + SI] adds the base address in BX and the index in SI to calculate the effective memory address, and then moves the value from that address into the AX register. Another example, ADD BX, [DI + 20h], adds the value stored at the memory address formed by the sum of the DI register and 20h to the BX register.

The flexibility in addressing modes makes the x86 architecture adaptable for various programming scenarios, particularly in systems-level programming and performance optimization.

1.2.1.3. Instruction Set Overview

The instruction set of x86 architecture is known for its CISC design. CISC processors like x86 can execute complex instructions that combine multiple low-level operations into a single instruction. Here are some categories of instructions that form the backbone of x86 processors:

Data transfer instructions: These instructions handle the movement of data between registers, memory, and I/O devices. Examples include MOV for moving data, PUSH and POP for stack operations, and IN and OUT for input/output transfers throughout streams.

Arithmetic instructions: These instructions perform basic mathematical operations such as addition, subtraction, multiplication, and division. Common instructions include ADD, SUB, MUL, and DIV.

Bitwise operations: Instructions like AND, OR, XOR, NOT, and SHL (shift left) manipulate individual bits within a frame of a word, enabling efficient low-level data processing.

Control flow instructions: These instructions change the flow of execution within a program, allowing for jumps and branching. Examples include unconditional jumps (JMP), conditional branches like JE (jump if equal) and JNE (jump if not equal), as well as procedure calls (CALL) and returns (RET).

String manipulation instructions: The x86 architecture includes specialized instructions for processing strings. Instructions like MOVS, CMPS, SCAS, and LODS allow efficient string operations.

System control instructions: These instructions manage the processor's state and interactions with hardware. Examples to list are CLI (clear interrupt flag) and STI (set interrupt flag) for interrupt control, as well as HLT for halting processor operations.

1.2.1.4. Memory Segmentation in x86 Architecture

One of the distinctive features of the original x86 architecture is its use of memory segmentation. While more modern systems (including x64) have largely transitioned to a flat memory model, segmentation is still supported and was fundamental to early computing:

Segment Registers: The x86 architecture utilizes six segment registers (CS, DS, SS, ES, FS, GS) to partition memory into segments for code, data, and stack. This segmentation enables more efficient memory management, particularly in systems with limited address space.

Segmented Memory Model: In the segmented memory model, addresses are represented as a combination of a segment and an offset. This allows the CPU to access a larger memory space than a single register can handle on its own. For instance, in real mode, a 16-bit register can address up to 64 KB of memory per segment, enabling the system to manage more memory efficiently.

1.2.1.5. Interrupts and Exception Handling

Interrupts and exceptions are key components of the x86 architecture, allowing the processor to react to both hardware and software events:

Hardware interrupts: These are signals generated by external devices, such as keyboards or disk drives, that notify the CPU to manage time-sensitive tasks, like handling input or processing data transfers.

Software interrupts: Triggered by programs using the INT instruction, software interrupts allow applications to request specific services from the operating system, facilitating communication between software and hardware.

Exceptions: These are a special type of interrupt triggered by the processor itself when it detects an error, such as division by zero or invalid memory access. Handling exceptions is crucial for maintaining system stability.

1.2.1.6. Protected Mode and Real Mode

The x86 architecture supports multiple operating modes that govern how memory and instructions are handled by the processor:

Real mode: This is the basic operating mode in which the processor starts upon booting. It allows direct access to memory and hardware, but is limited to 1 MB of addressable memory and does not support modern features like multitasking or virtual memory.

Protected mode: Introduced with the 80286 processor, protected mode removes the 1 MB memory limit and introduces features like virtual memory, multitasking, and memory protection. It ensures that one program cannot overwrite the memory used by another, thus enhancing system security and stability of the system.

Virtual 8086 mode: This mode allows modern processors to run programs designed for real-mode (such as DOS programs) while still in protected mode, using hardware virtualization to isolate and manage legacy applications.

System Management Mode (SMM): SMM is a specialized operating mode used for managing critical system-wide tasks, such as power management, hardware control, and secure execution of system-level code outside the regular operating environment.

Evolution of x86 - from 16-bit to 32-bit and 64-bit: The x86 architecture began with 16-bit processors but saw a major leap with the introduction of the 80386, which expanded the architecture to 32 bits. This transition greatly increased the addressable memory and processing capabilities, enabling the execution of more complex applications and operating systems. Later, the evolution continued with the expansion to 64 bits, further enhancing performance and memory capacity.

With the transition to 64-bit processors (x86-64 or AMD64), the architecture could address far more memory (up to 16 exabytes), making it ideal for modern applications that require high-performance computing. The x86 architecture's ability to evolve while maintaining backward compatibility has contributed to its widespread adoption and longevity.

In short, the x86 architecture is one of the most versatile and long-lasting architectures in computing history. Its design, featuring a complex instruction set, extensive addressing modes, memory segmentation, and multiple operating modes, has allowed it to adapt to the ever-growing demands of modern computing. Understanding the x86 architecture is crucial for anyone working in fields such as systems programming, reverse engineering, or cybersecurity, as it forms the foundation of much of the software and hardware in use today.

1.2.2. x64 Architecture

The x64 architecture is an extension of x86 that remains backwards-compatible. It introduces a new 64-bit mode while retaining a legacy 32-bit mode that functions the same as x86.

The term "x64" refers to both AMD64 and Intel64, which have nearly identical instruction sets.

In x64, the original 8 general-purpose x86 registers are expanded to 64 bits, and 8 additional 64-bit registers are introduced. These 64-bit registers are named with an "r" prefix, such as rax, the 64-bit version of eax. The newly added registers are named r8 to r15.

The lower 32-bit, 16-bit, and 8-bit sections of each 64-bit register are also directly accessible. This includes registers like esi, which did not previously have an accessible lower 8-bit portion. A table typically specifies the assembly language names for these lower parts of the 64-bit registers. The following table provides the assembly-language names used to access different portions of the 64-bit registers in x64:

Table 1.1. Names for different portions of 64-bit registers

Function	Register	32-bit	16-bit	8-bit high	8-bit low
Accumulator Register	RAX	EAX	AX	AH	AL
Base Register	RBX	EBX	BX	BH	BL
Count Register	RCX	ECX	CX	CH	CL
Data Register	RDX	EDX	DX	DH	DL
Source Index	RSI	ESI	SI	-	SIL
Destination Index	RDI	EDI	DI	-	DIL
Base Pointer	RBP	EBP	BP	-	BPL
Stack Pointer	RSP	ESP	SP	-	SPL
Additional general-purpose registers	R8	R8D	R8W	-	R8B
	R9	R9D	R9W	-	R9B
	R10	R10D	R10W	-	R10B
	R11	R11D	R11W	-	R11B
	R12	R12D	R12W	-	R12B
	R13	R13D	R13W	-	R13B
	R14	R14D	R14W	-	R14B
	R15	R15D	R15W	-	R15B

Another significant enhancements in the x64 architecture is its ability to address a much larger memory space. While x86 processors are limited to

addressing 4 GB of RAM due to their 32-bit address space, x64 processors can theoretically address up to 16 exabytes of memory (though actual implementations typically allow up to several terabytes). This increased memory capacity is crucial for applications such as high-performance computing, virtualization, databases, and other memory-intensive workloads.

While x86 relies on memory segmentation to manage memory, x64 largely eliminates the need for segmentation. In x64 architecture, a flat memory model is employed, where memory is accessed through a unified 64-bit linear address space. While segment registers like CS, DS, and SS are still present, they are seldom used in most applications, simplifying memory management and enhancing efficiency.

This shift simplifies the memory model, making the architecture more efficient and less prone to errors related to memory segmentation. However, the segment registers are still available for compatibility with legacy software.

One of the major strengths of the x64 architecture is its backward compatibility with x86 code. x64 processors can run both 32-bit and 64-bit programs, allowing users to continue using legacy software while benefiting from the performance improvements of a 64-bit environment. In long mode (the 64-bit mode of x64), processors can seamlessly switch between executing 64-bit and 32-bit code.

The processor operates in two primary modes:

Compatibility mode: Allows execution of 32-bit x86 code within a 64-bit operating system.

Long mode: The native 64-bit mode, where the full benefits of the x64 architecture are realized.

In addition to supporting larger physical memory, x64 processors also feature enhancements in virtual memory management. x64 uses a 4-level paging scheme, allowing access to larger virtual address spaces (up to 256 TB). This is

essential for operating systems that need to manage large datasets or multiple applications concurrently.

The expanded address space in x64 architecture enables stronger memory protection features, such as Data Execution Prevention (DEP), which blocks code execution in specific memory regions, and Address Space Layout Randomization (ASLR), which alters the memory layout of programs to help defend against various types of attacks.

In general, the x64 architecture represents a significant evolution from the 32-bit x86 architecture, offering increased memory capacity, additional registers, and enhanced performance capabilities while maintaining backward compatibility with existing x86 applications. Its expanded instruction set, simplified memory management, and support for advanced features like virtualization make it the architecture of choice for modern computing environments. With x64 processors now ubiquitous in everything from personal computers to data centers, understanding its key features and differences from x86 is essential for software developers, system architects, and security professionals, especially malware researchers alike.

1.3. PE File Format

Modern operating systems use standardized file formats to define how executable code and associated data are stored on disk. One such format is the Common Object File Format (COFF), initially developed for Unix-like systems and used for executables, object code, and shared libraries. An extension of COFF, the Portable Executable (PE) format, is primarily used in Windows operating systems for files such as executables, object code, DLLs, FON font files, and core dumps. It supports both 32-bit and 64-bit Windows platforms, serving as a versatile format for various file types.

The PE format functions as a data structure that provides the Windows operating system loader with essential information for executing the contained code. This includes references to dynamic libraries, API exports and imports,

resource management details, and thread local storage (TLS) information, which is vital for running multithreaded applications efficiently. The PE file format is flexible and supports many file types, with extensions such as .exe for executables, .dll for dynamic link libraries, .scr for screensavers, and .sys for system files, being just a few examples.

The PE file format consists of several essential components that work together to manage how executable code is loaded into memory. These structures act as a bridge between the file's on-disk representation and its in-memory execution by the Windows operating system. A key aspect of the PE format is that the data structures used on disk are the same ones used in memory. This means that if you can locate a specific piece of data in the PE file on disk, you can easily find the same information once the file is loaded into memory.

However, the PE file is not loaded into memory as a single block. The Win32 loader selectively maps portions of the file into memory based on which parts are necessary for execution. This process optimizes memory usage and performance by loading only what is required, such as executable code, imported libraries, and resources. Some parts of the file, like relocation information, may be read without being mapped, while sections like debug information may not be loaded into memory at all if they aren't needed for execution.

When an executable file is loaded into memory, it is represented as a module containing all the necessary code, data, and resources required for the process to run. Each module is self-contained, including executable code and references to libraries that the program relies on.

A critical field within the PE header instructs the system on how much memory to allocate for the executable. This allocation covers all the parts of the file that will be mapped into memory for execution. Portions of the file that are not needed during execution, such as relocation or debugging data, are typically placed at the end of the file, beyond the sections that will be mapped. This helps

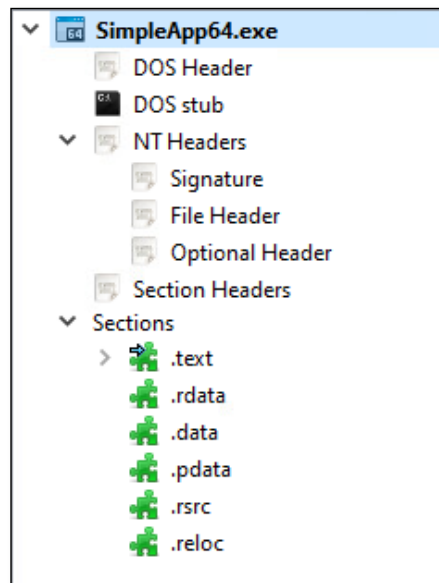


Figure 1.3. Structure of a PE file

DOS Header

Every PE file starts with a structure called the DOS header, which is 64 bytes long. This header makes the file recognizable as an MS-DOS executable. Although modern Windows programs no longer rely on MS-DOS, the DOS header remains in the PE format to ensure compatibility with earlier systems. It contains information, including a pointer to where the PE header begins.

DOS Stub

Immediately following the DOS header is the DOS stub. This is a small, MS-DOS 2.0-compatible program embedded in the file. If the file is accidentally run in DOS mode, the DOS stub executes and displays a message that reads, “This program cannot be run in DOS mode.” This stub ensures that the user is notified that the executable is not compatible with DOS, though it's largely obsolete in modern Windows environments.

NT Headers

The NT Headers part contains three main parts:

PE signature: This 4-byte value identifies the file as a PE file. It serves as a flag for the Windows operating system to treat the file as a PE format executable.

File Header: Also known as the COFF header, this component provides basic information about the PE file, such as the target architecture (e.g., x86, x64),

the number of sections in the file, and timestamps indicating when the file was created. This header also includes flags that describe the nature of the file, such as whether it is an executable or a DLL.

Optional Header: Despite its name, the Optional Header is required for executable files like .exe and .dll files, but not for object files. It contains critical data for the operating system loader, such as the preferred memory address (image base) for loading the file, the entry point address (where execution should begin), and the size of the executable in memory. This header also specifies subsystem details (e.g., GUI or console) and memory management settings like stack and heap size.

Section Table

Following the Optional Header is the section table, an array of image section headers. Each entry in the section table corresponds to a section within the PE file. Each section header provides details about its associated section, such as:

- **Section Name:** A short label (like ".text" for code or ".data" for initialized data).
- **Virtual Size:** The size of the section when loaded into memory.
- **Raw Data Size:** The size of the section as stored on disk.
- **Virtual Address:** The virtual address at which the section will be loaded into memory.
- **Characteristics:** Flags that indicate properties of the section, such as whether it's executable, readable, or writable.

Sections

The actual content of the PE file is stored within its sections. Each section serves a distinct purpose in the program's execution and contains different types of data:

- **.text Section:** Contains the program's executable code.
- **.data Section:** Stores initialized global and static variables.
- **.rdata Section:** Holds read-only data, such as constants and strings.

- .bss Section: Contains uninitialized data that will be allocated and initialized at runtime.
- .rsrc Section: Includes resources like icons, images, and strings that the program might use.
- .idata Section: Manages information about imported functions and libraries that the executable relies on.

When a PE file is executed, the Windows loader begins by reading the headers to determine how the file should be loaded into memory. The loader checks the Import Table to identify any dynamic libraries that the executable depends on. If necessary, the loader will load these libraries into memory and resolve any references to external functions or data. The Export Table may also be checked to ensure that any required functions provided by the executable or its associated DLLs are available for use by other programs.

The Section Table is consulted to determine how the different sections of the PE file should be mapped into memory. The .text section, containing the executable code, is loaded into an executable memory region, while the .data section is loaded into a writable memory region. The Resource Table may also be used to load resources such as icons or menus into memory.

Once the PE file has been fully loaded and all dependencies have been resolved, the loader transfers control to the entry point of the program, and execution begins.

The PE format is an integral part of the Windows operating system, offering a versatile and efficient measure for storing and loading executables, object code, and other resources. Its structure enables Windows to manage resources efficiently by mapping only the necessary portions of the file into memory, thus optimizing performance and reducing memory usage. The format's extensibility and backward compatibility ensure that it remains the standard for Windows executables, from simple applications to complex dynamic link libraries.

Understanding the internal structure of PE files is crucial for developers, malware analysts, and reverse engineers who need to work with or analyze executables at a low level.

1.4. Structure of Packed Code

In binary analysis, especially malware analysis, packed code refers to executables that have been compressed or encrypted to obfuscate their true purpose and behavior. This process, known as packing, is commonly used to evade detection and analysis by security tools and researchers. A packed code typically includes three main components: the PE (Portable Executable) header, the unpacking routine (or stub), and the packed payload. The unpacking routine is executed first, with the primary goal of decompressing or decrypting the packed payload and passing executable flow to the OEP, where the legitimate code execution starts.

Packing and unpacking process introduces a layer of complexity, making it harder to analyze the actual payload without first unpacking the code in memory. Various packers, such as UPX, create different structures for packed code. Understanding the fundamental structure of a packed binary is essential for unpacking and retrieving the original, unobfuscated binary that can then be analyzed for potential malicious behavior.

The figure below illustrates the packing and unpacking process, showing the transition from an original binary to a packed binary and, finally, to the unpacked binary in memory, after the execution of the unpacking routine.



Packing and Unpacking process

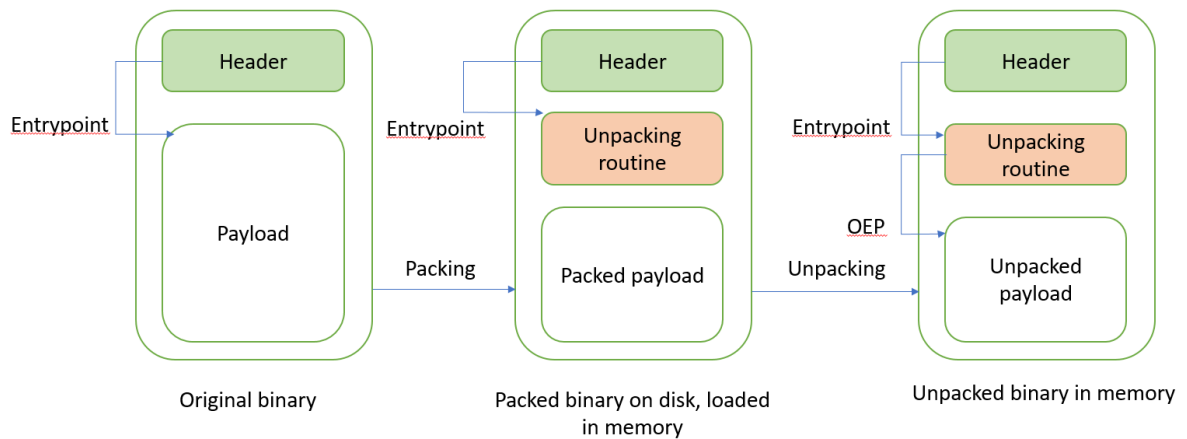


Figure 1.4. Packing and unpacking process

In general terms, a packed executable consists of a PE header, an unpacking code (stub), and the compressed payload. The structure of packed code can vary depending on the packer used. For example, UPX generates the packed file with the following structure:

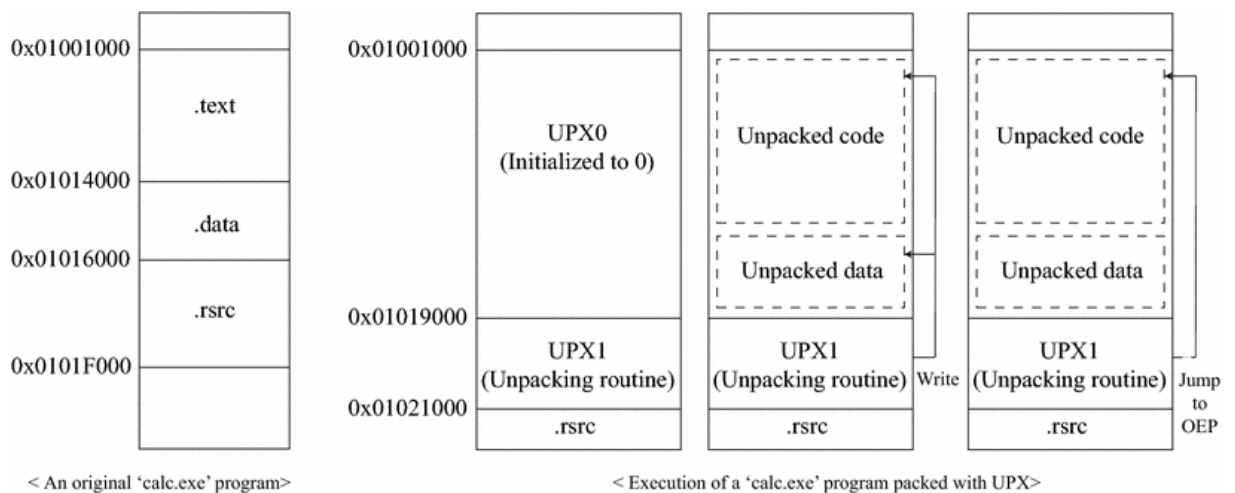


Figure 1.5. An example of UPX packed code

A packer compresses the payload, and when the packed executable is run, it begins by unpacking the compressed payload. The OEP marks the starting point of the unpacked payload. Our focus is on the unpacking code, which often employs obfuscation techniques that are largely unrelated to the actual content of the payload. The unpacking code typically ends just before reaching the OEP.

The unpacking code is a critical part of the packed executable. This section is responsible for restoring the compressed or encrypted payload back to its original form in memory, allowing the program to execute as intended. The unpacking process typically includes steps like:

- Decompressing or decrypting the packed payload.
- Restoring the PE sections of the original executable into memory.
- Redirecting the control flow to the OEP of the unpacked payload.

Packers often use obfuscation techniques within the unpacking code to hinder reverse engineering. These techniques may include:

- Control flow obfuscation, where the unpacking code's logic is altered to make static analysis more difficult.
- Dynamic code modification, where code is altered or generated at runtime, making the unpacking process harder to track.
- Anti-debugging measures, where checks are inserted to detect if the unpacking process is being analyzed by a debugger, potentially terminating the process or altering its behavior.

Despite these measures, the unpacking code typically follows a discernible flow as mentioned earlier: The unpacking process involves extracting the original executable, typically stored in additional sections of the packed file, into memory, followed by resolving all the original imports. Once this is completed, execution is transferred from the unpacking stub to the OEP.

Loading the executable:

The loading process is similar to the unpacking phase, as it involves formatting the PE header to ensure that any loader can successfully allocate memory for the executable sections. The unpacking stub is responsible for unpacking the code and copying it into the allocated memory sections. The specific method of accomplishing this task is generally defined within the unpacking stub itself.

Resolving imports:

The most common method for resolving imports is to utilize the *LoadLibrary* and *GetProcAddress* functions to obtain the addresses of imported functions. One straightforward approach is to leave the import table intact; however, this compromises the goal of obfuscating the imported functions. Alternatively, some packers retain only one function from each imported library to maintain a functional import table while still providing some level of obfuscation. A more complex method involves removing all imports entirely, which complicates the unpacking stub but can enhance obfuscation.

Tail jump:

Once the unpacking stub completes its tasks, execution must jump to the OEP. This instruction is known as the “Tail jump”. To obscure this jump, packers might replace it with a *ret* or *call* instruction. In some cases, they may also utilize operating system functions such as *NtContinue* or *ZwContinue* to achieve the same goal.

Differences from the original program:

It is important to note that the unpacked program still differs from the original executable, as it includes the unpacking stub and additional code introduced by the packer. The PE header is reconstructed in memory during unpacking, so it will not match the original version exactly.

Indicators of a packed program:

Several typical indicators can suggest that a program is packed: A minimal number of imported functions; little to no recognizable code when analyzed with a disassembler's auto-analysis feature. Sometimes, debuggers like OllyDbg may identify what appears to be an unpacking stub and provide warnings. Presence of sections with names commonly associated with packers, such as *UPX0*, or even arbitrary, random names.

Unusual section sizes, such as a 'Raw Data' size of 0, while the 'Virtual Size' is significantly larger.

High entropy in binary sections may indicate that the binary is packed and/or encrypted, particularly when this occurs in the binary sections themselves.

Common packers and their attributes:

UPX (Ultimate Packer for Executables) is one of the most frequently used packers. One of its key features is the ability to easily decompress packed files using the same tool with the -d option. However, malware developers often tweak UPX or opt for different packers, which makes the standard UPX decompression process ineffective. As a result, even if the file seems to be packed with UPX, the usual UPX -d method may fail to unpack it.

PECompact is another widely utilized packer, often incorporating anti-debugging mechanisms and obfuscated code to hinder reverse engineering efforts. To unpack PECompact, manual debugging is often needed to find the Tail Jump, which is typically a jmp eax instruction, followed by several 0x00 bytes. These null bytes are used to further obfuscate the process, requiring manual inspection to locate the Original Entry Point (OEP).

ASPack is notable for its use of self-modifying code, making the unpacking process more complex. If the program detects that it is being debugged, it often terminates prematurely. Due to its widespread use, many automated unpackers have been developed to handle ASPack, although their effectiveness varies with different ASPack versions. Typically, ASPack's unpacking stub includes a PUSHAD instruction, and a common technique to locate the OEP is by setting a hardware breakpoint on a 'read' operation for the stack addresses that hold the register's value.

Petite, like ASPack, implements anti-debugging strategies that complicate the analysis process. It employs 'single-step' instructions designed to disrupt debuggers and pass exceptions to the program, evading detection. Unpacking Petite is similar to ASPack, as it involves setting a hardware 'read' breakpoint on relevant stack addresses. Additionally, Petite tends to keep only one import per library in the import table, which can aid in the analysis.

WinUpack is recognized for concealing the Tail Jump within the unpacking stub, making it difficult to spot. A common characteristic of WinUpack is the use of push instructions followed by a ret. To identify the OEP, a useful method is setting a breakpoint at GetProcAddress and stepping through the code, specifically looking for loops that restore the import table. Another effective strategy is setting breakpoints at GetModuleHandleA or GetCommandLineA and tracing backward to find the OEP.

Themida stands out as one of the most complex and secure packers available today. It integrates numerous anti-virtual machine (VM) and anti-debugging measures, along with kernel-level code that continues to run even after the program has been unpacked. Due to its complexity, automated unpacking tools often fail to handle Themida, and a common approach is to dump the program from memory after partial unpacking.

Each of these packers employs a range of sophisticated techniques designed to impede reverse engineering and unpacking, requiring specialized methods for successful extraction and analysis.

1.5. Binary Analysis Difficulties

1.5.1. How Obfuscation Hinders Malware Analysis

The complexity of analyzing packed malware largely stems from the obfuscation techniques applied by packers. When a program is obfuscated, often through methods like encryption, it becomes difficult to analyze using static techniques. This allows the packed code to evade detection by firewalls and antivirus programs. According to the classification by BE-PUM authors, obfuscation techniques fall into six main categories:

- Self-modifying code (dynamic code): Involves the program rewriting parts of its own code at runtime, including methods such as packing/unpacking and overwriting sections of code.

- Entry/code placing obfuscation (code layout): Involves techniques like overlapping functions, overlapping code blocks, and breaking code into chunks (code chunking).
- Instruction obfuscation: Uses indirect jumps to obscure the flow of the program, making it harder to follow in disassembly or debugging.
- Anti-Tracing: Incorporates techniques such as SEH and the use of specific APIs like *LoadLibrary* and *GetProcAddress* from *kernel32.dll* to hinder tracing during analysis.
- Arithmetic operation obfuscation: Obfuscates constants and employs checksumming to complicate the reverse engineering process.
- Anti-tampering: Includes mechanisms such as timing checks, anti-debugging features, anti-rewriting techniques, and hardware breakpoints. Anti-rewriting involves the use of stolen bytes and checksumming to prevent code modification.

Notably, anti-tampering stands out as it involves virtual machine detection and trigger-based actions, which can interfere with dynamic analysis and monitoring tools.

- Anti-debugging: Detects the presence of a debugger through API calls, such as *kernel32.IsDebuggerPresent*.
- Stolen bytes: Allocates a memory buffer using *VirtualAlloc*, into which the unpacked code is written, often leaving gaps in the original code.
- Timing check: Compares the program's execution timing to that of a native Windows environment to identify anomalies.
- Hardware breakpoints: Stores jump destinations in debug registers like DR0, DR1, DR2, and DR3 to evade traditional debugging techniques.

These obfuscation techniques serve to complicate reverse engineering efforts and make malware analysis significantly more challenging.

Besides, there are several common obfuscation techniques include:

- Opaque predicates: Conditional branches that are designed to always resolve the same way but are difficult for static analysis to predict.
- Control-flow flattening: The insertion of unnecessary control structures to obscure the true flow of execution.
- Dynamic API resolution: Instead of statically linking to common APIs, the unpacking code resolves API addresses at runtime, making it harder to understand which system calls are being used.
- Code chunking: The unpacking process is split into multiple small, seemingly unrelated pieces of code, which are dynamically combined during execution.

1.5.2. Deep Inspection of API Obfuscation Techniques

Current methods for reconstructing import tables often depend on several assumptions that may not account for the intricacies of advanced packers. These assumptions include:

- The target API's address can be statically located within the unpacked code section;
- Control flow reaching a DLL must point directly to the target API's entry point;
- API calls must be routed through the IAT. Unfortunately, several studies and references give negative answers to all of them.

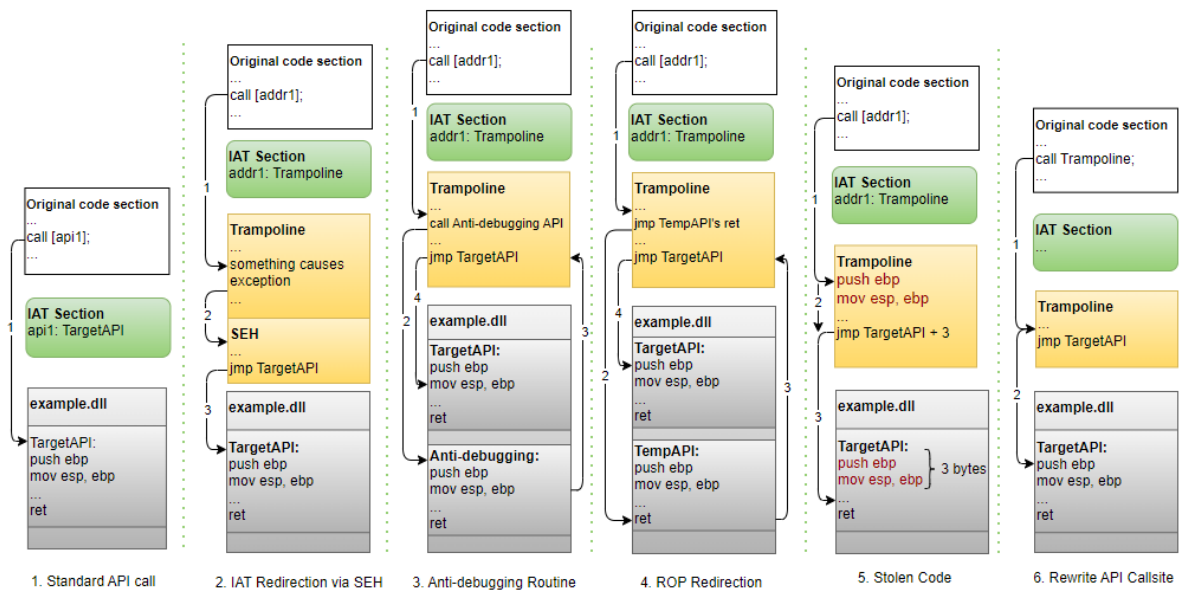


Figure 1.6. Illustration of different API obfuscation schemes

As discussed on page 6 [2], the first question is: ***Can the addresses of target APIs be statically identified in the unpacked code?*** In the diagram (Figure 2), the IAT entry first points to a "trampoline" area. This code section, maintained by the unpacking routine, serves as an intermediary to obfuscate the control flow of standard API calls. The trampoline installs a custom structured exception handler (SEH) and executes an intentionally faulty instruction (e.g., division by zero) to trigger the SEH, which is located elsewhere. The SEH then redirects the control flow to the target API. Without executing both the trampoline and SEH in Figure 2, it is impossible to determine the target API's address.

The second question is: ***Does control flow always point to a target API's entry point when it reaches a DLL?*** Dynamic-based techniques often assume that control flow reaching a DLL leads directly to the target API's entry point. However, there are several counterexamples that challenge this assumption, which can be grouped into three categories. First, certain packers (e.g., Armadillo, PEP, Obsidium) invoke anti-debugging APIs before reaching the target API, as seen in Figure 3. These APIs perform timing checks or checksums to thwart analysis. Second, some packers (e.g., PELock, Obsidium) employ a ROP-like

method to redirect API calls (Figure 4). In this case, the trampoline transfers control to the "ret-like" instruction of a temporary API before looping back to the trampoline and finally to the target API, a process known as "ROP redirection." Third, in the "stolen code" method, as shown in Figure 5, a few bytes from the beginning of an API are copied and executed before jumping back to the API's code after the copied instructions. Many packers, such as Themida, PELock, and Enigma, use stolen code to bypass API monitoring tools, which set hooks at API entry points. The large-scale analysis shows that stolen code usually copies 3 to 5 bytes or a single basic block from the target API, maintaining compatibility with the Position Independent Code (PIC) used in DLLs. Copying too many bytes could include relative addressing instructions, which might cause crashes during the execution time.

The third question is: ***Do API calls always go through the IAT?*** Most existing methods for reconstructing import tables assume that API calls must reference the IAT first. However, some packers (e.g., PEP, ASProtect, Themida) use direct call instructions to invoke target APIs without using the IAT. Figure 6 illustrates this approach, where packers rewrite the original indirect API call (opcode FF15) as a direct call (opcode E8). Since a direct call instruction is one byte shorter than an indirect call, packers also add a padding byte to ensure proper execution for the program.

All of the aforementioned techniques are hindrances to the process of rebuild the original executable payload.

1.6. Malware Analysis: Classical Methods and New Trends of Using Dynamic Symbolic Execution

Malware continues to grow in both complexity and volume. Over the past decade, significant advancements have been made in anti-malware strategies. As a result, malware analysis has become a vital aspect of cybersecurity, with researchers devising various techniques to study and counteract malicious software. Two primary approaches in this field are static and dynamic analysis.

Static analysis involves inspecting the malware's code without running it, enabling the detection of recognizable patterns, signatures, or heuristics. While this method is effective, it can face challenges when dealing with sophisticated malware that uses obfuscation or encryption to hide its behavior.

Dynamic analysis, on the other hand, involves running the malware in a controlled setting, like a sandbox, to monitor its behavior. This method is especially effective for identifying unknown or polymorphic malware, as it reveals the malware's real-time interactions with the system. However, dynamic analysis can be resource-heavy and might miss malware that changes its behavior when it detects it's being observed in a virtual environment.

An emerging approach in malware analysis is hybrid analysis, which combines both static and dynamic techniques to improve detection and classification accuracy. Additionally, machine learning and artificial intelligence are playing an increasingly vital role in this area, enabling the identification of new malware variants by analyzing vast datasets of malware behavior and signatures.

Researchers are also exploring memory forensics and network traffic analysis as complementary approaches to malware detection, focusing on identifying anomalous behavior within system memory or network communications. These advancements reflect the need to stay ahead of increasingly sophisticated threats in an ever-evolving landscape.

Building on the strengths of traditional static and dynamic analysis, symbolic execution introduces a deeper level of code exploration, and its advanced form, DSE, is increasingly becoming a critical tool in malware analysis. While static and dynamic methods provide valuable insights into a malware's behavior, they often struggle with heavily obfuscated or polymorphic malware that can evade simple signature-based detection. This is where symbolic execution offers a significant advantage, as it allows for a more exhaustive analysis of the code's potential behaviors:

Symbolic execution is a robust technique that provides a more in-depth examination of malware code than conventional analysis methods. Unlike standard static analysis, which reviews code without running it, or dynamic analysis, which monitors the execution of specific inputs, symbolic execution considers program inputs as symbolic variables instead of fixed, concrete values. This approach allows researchers to explore all possible execution paths a malware might take, depending on the conditions encountered during execution. As a result, it reveals hidden logic and behaviors that may remain undetected using conventional methods. For instance, symbolic execution can identify paths within the malware that only trigger under specific input values, such as those used to evade detection or trigger delayed payloads.

This comprehensive exploration contributes to building more informative and robust signatures for malware detection systems. Unlike traditional signatures, which focus on specific sequences of bytes or behavior, signatures derived from symbolic execution capture a wider range of potential behaviors, making them more resilient against polymorphic or metamorphic malware, which modifies its code to avoid detection. Symbolic execution also allows researchers to reason about how different parts of the malware interact with various inputs, such as network data, file system interactions, or user inputs, enabling them to discover conditional branches in the code that might remain dormant during conventional dynamic analysis.

A more advanced evolution of symbolic execution is DSE, which has emerged as a significant trend in malware analysis due to its ability to handle more complex, real-world malware. DSE enhances the capabilities of symbolic execution by combining it with runtime information. This hybrid approach allows the malware to execute while simultaneously applying symbolic analysis to certain parts of its code. By doing so, DSE uncovers execution paths that are often triggered by real-world inputs, such as those dependent on specific environments, configurations, or timing conditions. This makes DSE particularly effective

against malware that uses environment-sensitive behaviors, such as checking for the presence of a virtual machine or any debugger before loading, executing its malicious payload.

The integration of DSE into the malware analysis workflow bridges the gap between static and dynamic analysis techniques, allowing analysts to overcome the limitations of both. While static analysis is limited by its inability to observe runtime behavior, and dynamic analysis only explores one possible execution path, DSE provides a more complete picture of the malware's capabilities by exploring multiple potential paths simultaneously. This is especially valuable when dealing with highly evasive malware that employs techniques like code obfuscation, anti-debugging, or anti-virtualization. By analyzing both the symbolic execution paths and the concrete runtime behavior, DSE can reveal hidden functionality that might otherwise go unnoticed.

DSE also allows for more efficient analysis of malware by focusing symbolic execution on specific parts of the program where dynamic inputs play a crucial role, rather than attempting to symbolically execute the entire program. This targeted approach ensures that the analysis remains scalable, even for large and complex malware samples. Furthermore, DSE's ability to interact with real-world inputs means that it can more accurately model the actual behavior of the malware in different environments, providing insights that are critical for developing robust detection and mitigation strategies.

Incorporating DSE into malware analysis workflows offers a number of key benefits. First, it enhances the accuracy of detection by identifying execution paths that are specifically designed to evade traditional analysis methods. Second, it improves the comprehensiveness of the analysis, allowing researchers to uncover malware functionality that would otherwise remain hidden. Third, it helps in the development of resilient defenses, as the insights gained from DSE can be used to design more effective malware detection systems and better prepare defenses against emerging cybersecurity threats.

By addressing the weaknesses of static and dynamic analysis, DSE is becoming an indispensable tool for modern cybersecurity efforts. As malware continues to grow in complexity and evasiveness, techniques like DSE will play an increasingly critical role in keeping up with evolving threats, ensuring that organizations can detect and respond to even the most sophisticated malware attacks. In this way, DSE not only contributes to stronger malware defenses, but also helps to push the boundaries of malware research, providing a more proactive approach to cybersecurity that can anticipate and counter future attacks.

1.7. Conclusion

Chapter 1 provided a comprehensive overview of x86 and x64 architectures and also delved into the PE file format, offering a deeper understanding of how executables are structured in Windows environments.

Moreover, we examined the structure of packed code, emphasizing the relationship between the unpacking code, the packed payload, and the challenges they pose for reverse engineers. Additionally, the difficulties associated with binary analysis, particularly the various obfuscation techniques employed to hinder reverse engineering efforts, were discussed. The section on deep inspection of API obfuscation techniques underscored the growing sophistication of methods used to obscure executable behavior. In the last part of the chapter, we summarize some classical methods of malware analysis and a promising new direction in this area: DSE in malware analysis, which is the one that this thesis focuses on.

Together, these topics lay the groundwork for a deeper understanding of binary analysis and set the stage for the subsequent discussion on Dynamic Symbolic Execution in the next chapter.

Chapter 2. DYNAMIC SYMBOLIC EXECUTION

2.1. Symbolic Execution and Dynamic Symbolic Execution in Binary Analysis

Symbolic Execution (SE) and Dynamic Symbolic Execution (DSE) are both powerful techniques used in program analysis, particularly in the context of finding vulnerabilities, bugs, or understanding the behavior of a program.

Symbolic execution is a method that analyzes programs by treating inputs as symbolic values, rather than fixed, concrete values. Instead of running the program on specific inputs, SE explores all possible execution paths based on symbolic variables. The goal is to examine how the program behaves under various input scenarios without needing to provide real input values.

During symbolic execution, the system tracks the symbolic state of the program and generates a set of logical conditions, called path conditions, which define the constraints that must hold for the program to follow each specific execution path. By solving these conditions with a constraint solver, symbolic execution can determine whether certain program behaviors, like errors or crashes, are possible. SE is particularly useful in identifying potential bugs, vulnerabilities, and security issues by exploring a broader range of execution paths.

Key characteristics of SE:

- Uses symbolic inputs to represent variables.
- Explores multiple program paths in parallel, based on input constraints.
- Generates path conditions to represent each branch of execution.
- Can uncover hidden bugs or vulnerabilities across various input scenarios.
- Particularly effective for statically analyzing source code or binaries.

However, SE can encounter scalability challenges, especially when dealing with complex programs, because the number of possible execution paths can grow exponentially, leading to what's known as the "path explosion" problem.

Consider the following program:

```
def f (x, y):
    if (x > y):
        x = x + y
        y = x - y
        x = x - y
        if (x - y > 0):
            assert false
    return (x, y)
```

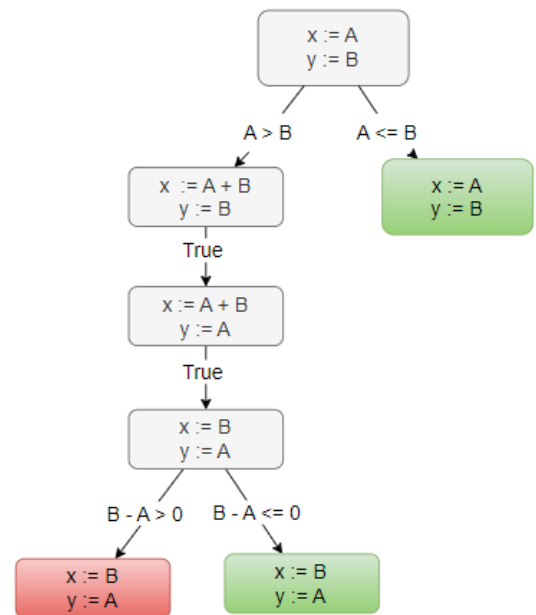


Figure 2.1. Symbolic execution example

Dynamic symbolic execution, often called "concolic testing" (a combination of concrete and symbolic), improves on traditional symbolic execution by combining both concrete and symbolic inputs during analysis. In DSE, the program is first executed with a specific, concrete input. Along with this concrete execution, symbolic execution is used to explore alternative paths that the program could take based on different inputs.

By merging concrete execution with symbolic analysis, DSE allows for deeper and more focused exploration of program paths. DSE monitors the program's execution with real inputs while maintaining symbolic expressions, which enables it to detect real-world vulnerabilities that would have been missed by pure symbolic execution.

In addition to path coverage, DSE is particularly useful in automated test generation, where it can automatically generate new test cases by exploring new execution paths. This capability makes DSE a powerful tool for vulnerability detection, especially in complex or obfuscated code where SE alone may struggle.

An SMT solver, such as Z3, is frequently employed to determine whether an explored execution path is feasible by checking the satisfiability of the path condition. However, traditional static symbolic execution can struggle with certain complexities, such as handling external code, resolving complex constraints, or managing indirect jumps in binary code. One approach to address these challenges is to use dynamic symbolic execution (DSE), which enhances the practicality of symbolic execution.

In symbolic execution, the next possible instructions are identified statically, and for each potential destination, an SMT solver is used to assess the feasibility of the newly generated path conditions. This process is efficient but can encounter difficulties with complex or dynamically determined code paths.

In contrast, dynamic symbolic execution (or concolic testing) dynamically checks the feasibility of the next instruction by executing the program with concrete input values derived from the symbolic preconditions. This often requires a binary emulator to evaluate the next instruction and determine the appropriate path to follow.

Considering a scenario where a pointer is dereferenced based on a dynamic value - a good example to illustrate its advantage over static symbolic execution involves handling pointer dereferencing in low-level code, such as C or assembly:

```

int* ptr;
if (x > 10) {
    ptr = &array[5];
} else {
    ptr = &array[2];
}
*ptr = y;

```

In the context of using SE, the executor would explore both branches of the if-else statement statically, generating two path conditions:

- For the path where $x > 10$, the path condition would be $x > 10$, and ptr would be assigned to $\&array[5]$.

- For the path where $x \leq 10$, the path condition would be $x \leq 10$, and ptr would be assigned to $\&array[2]$.

While SE can create these path conditions, it would need an SMT solver to analyze the symbolic memory addresses for ptr to determine the value being dereferenced. This process can become increasingly complex, especially if there are multiple levels of dereferencing or if ptr relies on external values.

In contrast, DSE would handle this situation more flexibly by executing the program with a concrete value of x . Let's assume $x = 15$ during one test case. The DSE engine would take this value and:

- Identify that the path $x > 10$ is the one being executed.
- Dynamically assign $ptr = \&array[5]$.
- Concrete execution would then dereference ptr and assign the value of y to $array[5]$.

DSE ensures that the exact memory location being modified by $*ptr$ is accurately captured through real execution, avoiding the need for complex symbolic analysis of pointer memory addresses.

When it comes to handling indirect jumps, for instance, if a function pointer or jump table is used to determine the next instruction:

```
void (*func_ptr)();
if (condition) {
    func_ptr = funcA;
} else {
    func_ptr = funcB;
}
func_ptr();
```

In static symbolic execution, the possible values for $func_ptr$ would have to be computed symbolically, and the solver would be required to explore each function's address manually. In contrast, dynamic symbolic execution would execute this code with concrete values for $condition$, dynamically setting $func_ptr$

and directly invoking either *funcA* or *funcB* during testing, making it easier to track complex, dynamic control flows in the program.

This dynamic resolution of jump destinations is a major strength of DSE, enabling it to handle intricate execution paths like function pointers or table-based jumps more effectively than purely static methods.

2.2. Introduction of BE-PUM

Binary Emulation for Pushdown Model generation (BE-PUM) is a tool for binary analysis, specifically designed to handle malware, which are typically small and obfuscated. BE-PUM takes an x86/Win32 executable binary as input and generates its CFG. To build the CFG, BE-PUM uses an on-the-fly symbolic execution approach, and this method is preferred for two key reasons:

At the binary level, data and instructions are handled similarly, meaning that the execution of a binary file can modify not only the data but also the instructions.

The current instruction and the environment during execution determine the next instruction's location. For example, in the case of an indirect jump like "jump eax", the next instruction is based on the value in the eax register at that moment.

BE-PUM uses symbolic execution to execute the input program. By employing concolic testing, BE-PUM determines the next instruction to process. A virtual simulation environment is crucial for managing stepwise execution throughout the process.

The BE-PUM system is implemented in Java. It utilizes JakStab 0.8.3 to disassemble the input binary file into assembly instructions. Additionally, the Z3.4.3 SMT solver is integrated as a backend to generate test instances for concolic testing.

BE-PUM's architecture comprises three main components: symbolic execution, binary emulation, and CFG storage. The system processes symbolic states by taking each state from the frontiers at the ends of explored execution paths. Symbolic execution then attempts to extend one step forward from the current state.

If the current instruction is an arithmetic operation (which updates the environment, and the next instruction's location is determined statically), BE-PUM disassembles the next instruction.

If a control instruction (such as a conditional jump) is encountered, concolic testing is used to determine the next location.

When a new CFG node or edge is discovered, this information is stored in the CFG storage, and the corresponding configuration is added to the frontiers. This process repeats until all branches are explored or an unsupported instruction, system call, or unknown address is encountered.

In execution, BE-PUM symbolically processes the input program using a path condition (pc) and an environment (Env), where pc refers to the symbolic value and Env is the mapping of variables to their respective values. These two components are handled independently within the implementation. Currently, BE-PUM supports around 250 x86 instructions, which were manually implemented by security researchers.

2.3. Introduction of Miasm

Miasm is an advanced reverse engineering framework that provides both static and dynamic binary analysis capabilities. Written in Python, Miasm is designed to facilitate the analysis of executable code by supporting various instruction sets and architectures, including x86 and x64. One of its standout features is its integration of symbolic execution, which allows for the exploration of program behavior through the use of symbolic variables.

In the context of binary analysis, Miasm leverages symbolic execution to uncover hidden paths within complex binaries. It is capable of translating binary code into an intermediate representation (IR), which serves as a powerful abstraction for both low-level and high-level analysis. This intermediate representation can then be used for symbolic execution, taint analysis, and other reverse engineering tasks.

Additionally, Miasm provides a robust API for extending its capabilities, making it highly customizable for specific use cases. Its ability to handle obfuscated code and its modular design make Miasm particularly effective for tasks such as malware analysis, vulnerability detection, and unpacking compressed or encrypted payloads.

By offering an array of tools for both static and dynamic analysis, Miasm plays a significant role in the field of reverse engineering. Its symbolic execution feature, in particular, is instrumental in tracing the flow of execution across different code paths, thereby enabling a deeper understanding of the program's logic and potential vulnerabilities.

2.4. Introduction of Triton

Triton is another powerful dynamic binary analysis framework that provides a suite of tools for performing symbolic execution, taint analysis, and dynamic analysis on binary code. Developed in C++ with Python bindings, Triton is designed to enable users to easily analyze 64-bit binaries, making it a versatile tool for both academic research and real-world applications.

A key feature of Triton is its symbolic execution engine, which allows for the analysis of a program's control flow and data flow. By representing program variables and memory addresses symbolically, Triton enables the exploration of multiple execution paths in parallel, providing a thorough examination of potential behaviors within the program. This symbolic exploration is particularly useful for uncovering vulnerabilities or reaching deeper parts of the code that might be difficult to access through traditional static analysis.

Triton's taint analysis feature further enhances its capabilities by tracking how data moves through a program, which is crucial for identifying security vulnerabilities, such as buffer overflows or unauthorized data access. The framework's dynamic nature allows it to analyze binaries in real-time as they execute, which provides a more accurate representation of how the code behaves

in practice, especially in the presence of obfuscation or anti-debugging, anti-tampering techniques.

Triton has been widely used in both research and industry for tasks such as vulnerability detection, malware analysis, and automated exploit generation. Its ability to integrate with existing debugging environments and its support for various instruction sets, including x86 and x64, make it a highly flexible and powerful tool for modern binary analysis.

2.5. Conclusion

Chapter 2 has provided a detailed exploration of DSE and its significance in modern binary analysis. By allowing the exploration of multiple program paths using symbolic variables, DSE proves to be a powerful technique for detecting vulnerabilities and understanding complex behaviors of binaries. Through the analysis of various tools, such as BE-PUM, Miasm, Triton, etc., this chapter has demonstrated how DSE is implemented and applied in different reverse engineering frameworks.

BE-PUM illustrates the use of symbolic execution in unpacking packed binaries, making it valuable when it comes to the area of malware analysis. Miasm, on the other hand, showcases its versatility in both static and dynamic binary analysis, providing users with an advanced framework for understanding and analyzing obfuscated code. Finally, Triton offers a robust combination of symbolic execution, taint analysis, and dynamic binary analysis, proving useful for tasks ranging from vulnerability detection to exploit generation.

Overall, this chapter highlights the growing importance of symbolic execution in the reverse engineering community. With its ability to uncover hidden paths and vulnerabilities, DSE has become an indispensable tool for security researchers, malware analysts, and software developers alike. In the next chapter, we will explore CFG and methods of OEP detection, building upon the foundations laid by DSE tool - BE-PUM.

Chapter 3. ROLES OF CONTROL FLOW GRAPH AND ORIGINAL ENTRY POINT DETECTION

3.1. Roles of Control Flow Graphs

Control Flow Graphs (CFGs) play an increasingly important role in malware analysis as modern malware continues to evolve at a rapid pace, rendering traditional signature-based detection methods less effective. Signature-based techniques rely on predefined patterns to identify known malware, but these methods fall short when faced with unknown or polymorphic malware, which can easily modify its appearance while maintaining its core malicious behavior. This is where CFGs come into play, providing a more flexible and comprehensive approach to analyzing and detecting malware.

A CFG is a graphical representation of all the possible paths that a program can take during its execution. In a CFG:

- Nodes represent individual instructions or blocks of code.
- Edges represent the transitions or jumps between these instructions based on the program's control flow, including conditional branches, loops, and function invocations.

By abstracting the execution paths in this way, CFGs allow analysts to understand the structure of a program independently of its specific code. This abstraction is particularly useful when analyzing malware because it captures the behavioral essence of the code, which can be much harder to obfuscate than simple bytecode signatures.

A key benefit of using CFGs in malware detection is their effectiveness in managing code obfuscation. Malware developers often use techniques such as encryption, packing, or polymorphism to disguise their malicious code. However, while the code itself may be obfuscated, the logical structure of the program, as represented by its control flow, often remains intact. This makes a CFG-based tool a powerful tool in detecting malicious behaviors even when the bytecode looks unfamiliar.

For instance, a packed malware may compress or encrypt its code to evade signature detection, but when unpacked and executed, its control flow will resemble that of a typical malicious payload.

In cases of polymorphic malware, where the code changes with each instance, the overall control flow patterns (e.g., sequences of loops and conditionals) may still reflect typical malicious operations such as payload delivery, data exfiltration, or privilege escalation.

By constructing and analyzing CFGs, malware analysts can look beyond superficial code changes to identify malware based on its fundamental behavior, rather than relying on fixed signatures.

As the complexity and volume of malware samples increase, manually analyzing CFGs becomes impractical. This has led to the application of machine learning techniques to automatically analyze and classify CFGs. In this context, machine learning algorithms can be trained to recognize the typical control flow patterns of malware by extracting features from CFGs and using them as inputs for classification models. These models can then distinguish between benign softwares and malicious ones totally based on their control flow graph characteristics.

For instance, a malware classifier may use features such as:

- Number of basic blocks: The number of distinct code blocks in a CFG can provide insights into complexity and purposes of malwares.
- Branching factor: The number of conditional branches and loops in a CFG can indicate suspicious control flows often seen in malware.
- Recursion and loops: Certain recursive or repetitive patterns are common in malware families.

By training on large datasets of malware and benign software, machine learning models can identify subtle but telling differences in control flow, enabling more accurate and scalable malware detection.

Detecting the OEP of a packed malware is one of the critical tasks in unpacking packed PE files. When malware is packed, the executable code is wrapped in a compressed or encrypted layer that makes it difficult to analyze. The OEP is the point at which the packed program finishes unpacking itself and begins executing its original malicious code.

In many cases, malware authors use packing techniques to obscure the OEP, making it challenging to determine where the actual malicious code starts. However, by analyzing the control flow graph of the unpacking stub, analysts can locate the OEP by identifying transitions in control flow that mark the end of the unpacking routine and the onset of the actual code execution.

In this thesis, we adopt the technique of graph similarity on CFGs to detect the OEP as presented in [\[4\]](#) with some improvements. This approach involves comparing the CFG of a packed executable with known templates or patterns of unpacked malware to identify where the code structure shifts from the unpacking routine to the malicious payload. By improving the accuracy of OEP detection, we can streamline the unpacking process, allowing for quicker and more efficient analysis of packed malware samples.

Beyond OEP detection, CFGs are used in a variety of advanced malware analysis techniques:

Automatic deobfuscation: CFGs can help reverse complex obfuscation techniques by identifying regions of code that perform redundant operations or loops intended to confuse analysts. By simplifying these sections, analysts can reveal the true intent of the code.

Control flow integrity: Control flow integrity is a security mechanism that leverages CFGs to ensure that a program's execution adheres to a predefined execution path. By enforcing that the program follows its intended CFG, control flow integrity can detect deviations, which may indicate control-flow manipulation attacks such as Return-Oriented Programming or Jump-Oriented

Programming. Any irregularities in the flow are flagged as potential security violations, providing a robust defense against these sophisticated attack methods.

Behavioral malware classification: CFGs allow for the classification of malware based on behavior rather than code similarity. By analyzing the control flow of different malware samples, researchers can identify common behaviors and categorize them into known malware families, even if their underlying code is different.

3.2. OEP Detection Method Based on Graph Similarity

3.2.1. OEP Detection

Detecting OEP is a crucial step in the process of unpacking and analyzing packed or obfuscated executables. The OEP is the point in the program's execution where the original, unpacked code begins after any packing or obfuscation routines have completed. Accurate detection of the OEP allows analysts to correctly reconstruct the executable's IAT and analyze the original, malicious code that was hidden by the packer.

Many modern packers, such as UPX, ASPack, and PECompact, etc., are used by malware authors to compress or encrypt their code, making it harder to analyze. These packing routines execute before passing control to the original code at the OEP. Hence, accurately identifying the OEP is critical for malware analysts to extract and analyze the original payload.

Several methods are commonly used to detect the OEP of packed executables. The choice of method often depends on the packing scheme employed by the malware.

Breakpoint method: The most straightforward method of detecting the OEP involves setting breakpoints on key API functions, such as GetProcAddress or VirtualAlloc, which are often used by unpacking routines. By allowing the unpacking code to run until it calls one of these functions, analysts can pause execution near the OEP and manually inspect the execution flow to identify the OEP of the executable payload.

For instance, using tools like x32dbg, analysts can trace the execution of the packed file, setting breakpoints at common unpacking-related APIs and manually stepping through the code to detect where the program jumps to the original entry point.

CFG analysis: CFG analysis is an advanced method used to detect the OEP by examining the program's control flow. By constructing a CFG for the unpacking routine and the original packed code, analysts can identify transitions from the unpacking routine to the original code. CFGs help pinpoint unusual jumps or transfers in control that typically occur at the OEP.

In this approach, tools generate a graph representing all possible execution paths within the program. By analyzing the graph's structure, the tool can detect shifts from complex unpacking routines to simpler, direct execution of the original code. This is especially useful in automated unpacking tools.

Heuristic-based OEP detection: Some tools use heuristics to detect the OEP based on the characteristics of packed executables. For example, after the unpacking routine completes, there is typically a noticeable change in the program's execution behavior. This could involve the program switching from decryption or decompression routines to standard code execution patterns, such as function calls, loops, or memory operations.

Heuristic-based approaches might scan for sequences like PUSHAD/POPAD, which are commonly found at the OEP of unpacked executables, or look for changes in memory protection, indicative of unpacked code being written to memory.

Pattern matching on known packers: Certain tools are designed to detect the OEP by recognizing specific patterns used by popular packers. Each packer typically generates unpacking code that follows a predictable sequence of instructions. By identifying these sequences, automated tools can detect the OEP and extract the original executable.

Malware researchers might be familiar with the tool Scylla or OllyDump, which may use predefined templates or signatures for packers like UPX, ASPack, or PECompact to automatically detect the OEP and begin IAT reconstruction.

While several methods exist for detecting the OEP, the task is not always straightforward. There are several challenges associated with OEP detection, especially in modern malware:

Multiple layers of packing: Some malware employs multiple layers of packing, where the program is packed several times. Detecting the OEP in such cases requires unpacking each layer sequentially, making the process time-consuming and complex.

Anti-debugging techniques: Malware often employs anti-debugging techniques to prevent reverse engineering tools from correctly identifying the OEP. These techniques may involve detecting breakpoints, altering execution flow when debuggers are present, or using timing checks to evade dynamic analysis methods.

Self-modifying code: Packed malware may include self-modifying code, where the code changes itself during execution. This complicates OEP detection because the original code may not be visible until after it has been modified.

Dynamic API resolution: Malware that dynamically resolves API calls at runtime (e.g., using hashes or lookup tables) can obfuscate the control flow and make it harder to detect the OEP using traditional methods. Such techniques often require more advanced analysis, such as symbolic execution or memory dumping, to reveal the OEP.

3.2.2. OEP Detection Based on Graph Similarity

Graph similarity techniques have emerged as a powerful tool for detecting the OEP in packed executables. This approach leverages the structural properties of CFGs to identify patterns indicative of the transition from unpacking routines to the original executable code. The premise of graph similarity in OEP detection

is based on comparing the CFG of a suspect binary against a database of CFGs from known unpacked binaries.

This method consists of the following steps:

CFG construction/generation: Initially, a CFG is constructed for the entire packed executable. This graph consists of nodes that illustrate basic code blocks, with edges indicating the control flow paths, such as jumps, function calls, and returns.

Graph feature extraction: Key features of the graph, such as node degrees, edge configurations, and subgraph patterns, are extracted. These features often capture the essence of the program's structure, which can distinguish between unpacking logic and the actual payload.

Template matching: The extracted features from the suspect binary's CFG are then compared to those of pre-analyzed unpacked binaries using graph similarity algorithms. This comparison seeks to find a match or close resemblance between the graphs, suggesting a similar pattern from unpacking code template to the one of the testing sample.

Similarity scoring: Algorithms such as graph kernels or edit distance are used to score the similarity between the CFG of the packed binary and each template CFG. A high similarity score suggests that this template belongs to the one of a certain packer and the point of transition in the suspect binary corresponds to the OEP.

Threshold determination: A threshold is set to determine when a similarity score is sufficient to declare a match. This threshold is often based on empirical data and may vary depending on the specific characteristics of the packers involved in the file.

One of the primary limitations of using graph similarity for OEP detection is its computational intensity, as calculating similarities between large control flow graphs can be resource-heavy and time-consuming, especially in complex binaries. Additionally, this method relies heavily on the availability of an extensive and up-to-date template database of CFGs from known unpacked

binaries. Without a comprehensive set of templates, the method may fail to accurately detect the OEP, particularly when dealing with new or custom packing techniques. Moreover, there is a risk of generating false positives or false negatives, especially if the packer employs novel or highly unique unpacking routines that do not closely match any existing templates. Despite these challenges, graph similarity remains a powerful approach for handling sophisticated packing and obfuscation techniques.

3.2.3. *Graph Similarity Using Weisfeiler-Lehman Kernel*

The Weisfeiler-Lehman (WL) graph kernel is a family of graph kernels used primarily for graph classification tasks. These kernels leverage the Weisfeiler-Lehman test of isomorphism, an algorithm used for distinguishing non-isomorphic graphs. This test works by iteratively updating the labels of nodes in a graph based on the labels of their neighboring nodes. The resulting sequence of graphs, each with refined labels, encodes progressively more detailed structural information about the graph. The WL graph kernel exploits this sequence to measure the similarity between graphs, making it an efficient tool for tasks such as graph classification.

The key idea behind the WL Kernel is to iteratively refine the representations of nodes (and their neighborhoods) in a graph by considering their neighbors' labels, and then using these updated labels to compute a kernel that measures the similarity between two graphs:

Weisfeiler-Lehman test of isomorphism: The Weisfeiler-Lehman test is a heuristic algorithm used to determine whether two graphs are isomorphic. In each iteration, the algorithm refines the node labels by considering both the node's current label and the labels of its neighbors. The process is repeated until the node label sets stabilize or differ between the two graphs. The WL test can distinguish many, though not all, non-isomorphic graphs and has become a fundamental tool in graph-based machine learning.

WL label refinement: The key idea behind the WL kernel is to generate a sequence of graph labelings, where each graph is a refined version of the previous one. Initially, each node is assigned a label. In each iteration, a new label is computed for each node based on its own label and the labels of its neighboring nodes. These new labels capture more information about the graph's structure as the iterations progress. After each iteration, the graphs are compared using kernel functions that measure the similarity of node labels.

Graph kernels: Graph kernels are functions that compute the similarity between graphs by comparing their structural features, such as subgraphs, walks, or paths. The Weisfeiler-Lehman kernel is part of a broader family of graph kernels, known as R-convolution kernels, which work by decomposing graphs into substructures (e.g., subtrees) and comparing them across graphs. The WL kernel achieves this decomposition through the iterative label refinement process.

Weisfeiler-Lehman subtree kernel: One of the most widely used instances of the Weisfeiler-Lehman kernel is the subtree kernel, which counts common subtree patterns in two graphs. The intuition is that graphs that share many subtree patterns are likely to be similar. This kernel has two main advantages: First, it scales linearly with the number of edges in the graphs and the length of the Weisfeiler-Lehman sequence, making it computationally efficient for large graphs. Second, it can handle both labeled and unlabeled graphs, making it applicable to various domains, such as bioinformatics and social network analysis.

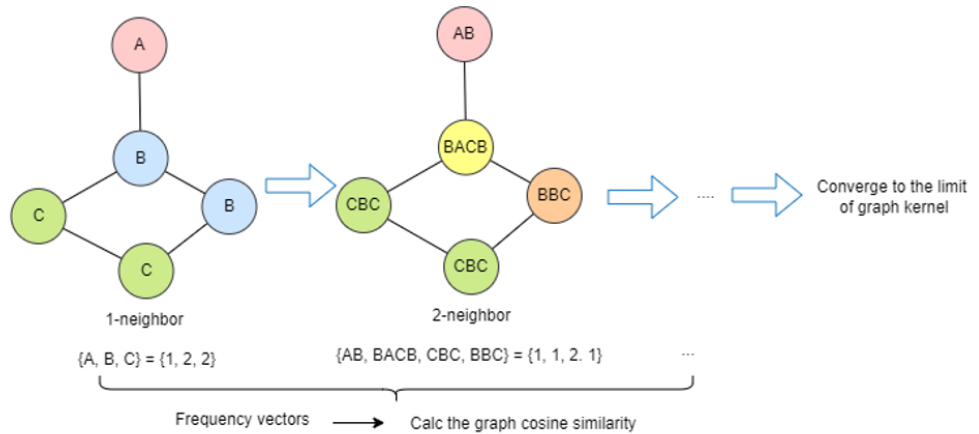


Figure 3.1. Illustration of Weisfeiler-Lehman Kernel algorithm

3.2.4. Control Flow Graphs of Unpacking Stubs

We currently agree with the idea presented in [4] that packed code generally shares a similar unpacking stub, regardless of the specific payload. Building on this, we explore the hypothesis that different packers produce unpacking stubs with distinct classes of CFGs. If this holds true, researchers could determine both the packer and its corresponding unpacking stub by using graph matching techniques. However, it's important to recognize that the graph matching process won't be exact, as even CFGs within the same class can have different offsets, leading to variations in the binary code. To mitigate this, we use graph similarity measures - specifically cosine similarity on Weisfeiler-Lehman histogram vectors - after normalizing the labels by stripping away arguments from the obtained instructions.

When both the unpacking stub and the original payload are identified, the unpacking stub's body can be pinpointed by finding the difference between the memory image after the packed code's execution and the original payload. In theory, the unpacking stub's CFG is represented by the predecessor graph at the stub's exit point. However, there is sometimes an unexpected path from the unpacked payload back to the unpacking stub. This is unusual since the original payload should not have prior knowledge of the unpacking stub. This occurrence is explained by the fact that both the unpacking stub and the unpacked payload can end up calling the same API.

- ***Forming the template for each packer***

The clustering procedure for setting up a template for a fixed packer involves five steps, using the CFG of packed code generated by BE-PUM. First, directed acyclic graphs (DAGs) of the unpacking stub are generated. Next, the Weisfeiler-Lehman histogram vectors for these graphs are computed. To ensure consistent vector dimensions, 0-padding is applied. Following this, the DBSCAN clustering algorithm is used with cosine similarity and an initial epsilon (eps) value of 0.05. Finally, the average Weisfeiler-Lehman histogram vector in each

cluster is paired with the corresponding end sequence, provided that the sequence is consistent.

If, during the DBSCAN process, some clusters exhibit inconsistent end sequences, we have to reduce the eps value by 0.01 increments and repeat the clustering until stabilization is achieved.

- **Comparing the templates to check the packer name and specify OEP**

When encountering an unknown packed code, the template-matching process involves four steps using the CFG generated by BE-PUM. First, during an incremental DFS trace of the CFG, retreating edges are removed, and a predecessor graph is generated at each node. Second, the Weisfeiler-Lehman histogram vector is computed for each predecessor graph. In the third step, if the end sequence of the predecessor graph matches a template, the similarity between their Weisfeiler-Lehman histogram vectors is checked. Finally, the template with the highest similarity is selected. At this stage, the node (sink node) is recognized as the exit of the unpacking stub, and the OEP is identified as the jump destination. Simultaneously, the packer used is also determined.

3.3. Conclusion

In this chapter, we explored the critical role of the CFG in binary analysis, particularly in identifying the OEP of packed executables. The CFG provides a structured way to visualize the execution paths within a program, making it a fundamental tool for understanding the flow of control during unpacking processes.

We then examined and adopted a method for OEP detection based on graph similarity focusing on the use of the Weisfeiler-Lehman Kernel for graph comparison from [\[4\]](#). This approach enables efficient matching of control flow patterns, allowing for the accurate identification of the OEP even in the presence of obfuscated code.

After understanding how the method addressed the use of template matching techniques for identifying specific packers (represented by templates

consisting of CFGs and sets of end-of-unpacking instructions) and detecting the OEP, which is particularly useful in automating the unpacking process, we will have a foundation to conduct more research about the next step of unpacking.

Through these measures, Chapter 3 provided a comprehensive overview of how graph-based techniques can enhance the accuracy and efficiency of OEP detection, ultimately aiding in reverse engineering efforts for packed and obfuscated binaries.

Chapter 4. IMPLEMENTATION AND EXPERIMENTS

4.1. Implementation

4.1.1. Data Preparation Module

We also use the packing dataset including benign x86 PE files and their packed versions grouped by packer name taken from GitHub repository [7] along with over 200 x86 malwares of all categories downloaded from <https://bazaar.abuse.ch/browse.php?search=tag%3Ax86>

Firstly, we have Ogawa Laboratory prepared CFGs of the whole dataset using **BE-PUM** with a duration of 500 seconds and got the CFG for over 700 benign samples and over 200 malware ones.

Secondly, we try to use **angr** to gen CFGs and make a comparison with the former method. Unfortunately, the result is not as expected. With 1456 packed files (taken from Git Hub pages), we successfully generated 179 CFGs, accounting for 12.29%. Therefore, we chose the result of BE-PUM (log file for packer identification comparison, and .dot file (CFG files) for OEP detection) as the input of the application.

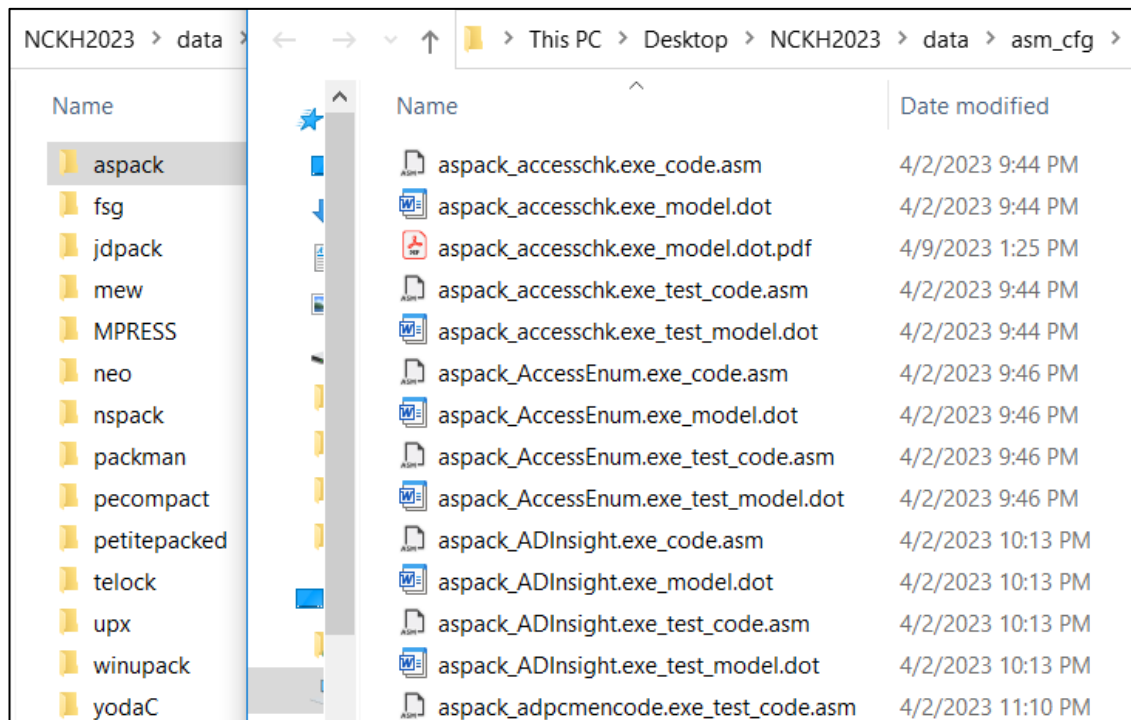


Figure 4.1. Dataset for training and testing packer identification and original entry point detection

We will discuss a little about the difference of these two DSE tools BE-PUM and angr:

Angr is a popular open-source framework for binary analysis that supports a variety of techniques, including dynamic symbolic execution, backward slicing, and data-dependency analysis.

Intermediate representation

Angr converts native binary code (such as ARM, MIPS, PPC, x86, and amd64) into an intermediate representation known as VEX IR. Besides VEX IR, angr also supports other representations, such as Capstone. It processes the binary by loading basic blocks, which are groups of VEX IR instructions.

CFG and indirect jumps:

Angr provides multiple methods for generating CFGs from binary files, and it can create two types of CFGs: CFGFast (static) and CFGEmlated (dynamic symbolic execution).

CFGFast is built using static analysis, making it much faster but less detailed in terms of control flow. This method is similar to those used by reverse-engineering tools like IDA Pro, Ghidra, and Radare.

CFGEmlated constructs the CFG through symbolic execution, aiming for higher accuracy. However, it is significantly slower and often incomplete due to issues such as missing system calls and hardware support.

The construction process begins with the analysis of basic blocks, which are groups of instructions that have a single entry and a single exit point. CFGEmlated first processes these basic blocks and adds direct jumps (or edges) between them. These direct jumps are easily resolvable because the target addresses are explicitly specified in the instructions themselves (e.g., jmp or call instructions to a fixed address). However, not all jumps in a program are direct. Many binaries, especially obfuscated or optimized ones, use indirect jumps, where the jump target is not explicitly given in the instruction and may depend on

runtime values or complex control flow mechanisms (e.g., jumps through a register or memory address space).

For indirect jumps, which can't be directly resolved, CFGEmulated uses a more sophisticated approach. It traverses backwards from the point of the indirect jump to identify a merge point - a point in the control flow where multiple paths converge. This is important because such points often represent decisions or computations that affect the jump target. Alternatively, if no merge point is found within a reasonable limit, CFGEmulated imposes a threshold based on the number of basic blocks analyzed, after which it begins symbolic execution.

Once the merge point is found, CFGEmulated uses symbolic execution to resolve the target of the indirect jump. Symbolic execution involves treating certain variables (like registers or memory values) as symbolic rather than concrete. By simulating the execution path symbolically, the tool can model possible values for those variables without having to run the program. To actually determine the value of the indirect jump target, the tool applies a constraint solver, which uses the conditions and constraints gathered from the symbolic execution path to infer the possible target addresses of the indirect jump.

This combination of control flow traversal, symbolic execution, and constraint solving allows CFGEmulated to handle complex control flow structures that involve indirect jumps. These could occur in various scenarios such as virtual function calls, switch-case constructs, or obfuscated control flows in malware or packed binaries. This makes CFGEmulated particularly powerful in reverse engineering scenarios where control flow isn't straightforward and needs advanced techniques to reconstruct accurately.

External call

One of the main challenges of using angr for analyzing real-world software is its environment model. angr models certain external calls using custom Python implementations called SimProcedure. However, it's not practical to have a SimProcedure for every library function, which means angr often ends up

executing statically loaded binary code and encounters issues with unsupported system calls. While angr provides a framework that allows users to hook into specific points and return custom values for function calls, this becomes a complex task for large binaries, especially when dealing with malware that typically makes over 100 external function calls.

4.1.2. Template Setup Module

We establish a template to represent a packer by using a pair of distinct but related items. The initial element is the average Weisfeiler-Lehman histogram vector for the packer cluster. This vector is obtained by using the Weisfeiler-Lehman graph kernel algorithm on the CFGs of binaries within the same packer cluster. By computing and averaging these vectors, we can capture the structural characteristics that are common to binaries packed with the same packer. The Weisfeiler-Lehman algorithm is useful in this context because it generates graph embeddings that are robust to small variations in structure, which is crucial when dealing with different versions or slightly modified outputs from the same packer.

The second item in the template is the final sequence of the unpacking stub, which represents the end instructions executed by the stub before transferring control to the original, unpacked code. This sequence is included in the template if it is consistent across binaries in the same cluster. The unpacking stub is the part of the packed binary that performs the actual unpacking of the payload, and identifying its end is crucial for correctly recovering the OEP. To ensure that the end sequence is representative of the entire cluster, we analyze whether this sequence remains the same (i.e., consistent) across all binaries in the cluster.

To achieve this consistency, we need to fine-tune the clustering algorithm, particularly by adjusting the epsilon (eps) value used in the clustering process. The epsilon value determines how tightly or loosely the clustering algorithm groups the binaries together. If the epsilon value is too large, binaries that are too different might end up in the same cluster, leading to inconsistent end sequences. Conversely, if the epsilon value is too small, the clustering might be too strict, and

binaries that are essentially packed with the same packer but have minor variations might be separated into different clusters. By carefully adjusting the epsilon value, we can ensure that binaries with a consistent end sequence of the unpacking stub are grouped together, allowing for a more reliable template to be generated and used.

This template, comprising the average frequency vector of nodes and the consistent set of unpacking stub ending instructions, serves as a signature for identifying binaries packed with a particular packer. It provides an effective way to recognize packed binaries in future analyses by comparing their CFGs and unpacking stubs to the template, aiding in tasks such as malware detection, reverse engineering, and unpacking automation.

- Clustering packers:

```

X = np.asarray(X)
eps_default = 0.3
while True:
    print("esp = {}".format(eps_default))
    if eps_default <= 0:
        break
    clustering = DBSCAN(eps=eps_default, min_samples=2, metric="cosine").fit(X)
    labels = clustering.labels_
    cluster_by_idx = {}
    for idx, packer_name_iter in enumerate(packer_name_list):
        print("packer name: {}, {}".format(packer_name_iter, labels[idx]))
        if not (labels[idx] in cluster_by_idx):
            cluster_by_idx[labels[idx]] = []
        cluster_by_idx[labels[idx]].append(packer_name_iter)

    not_cross = True
    nb_of_class = 0

    for key in cluster_by_idx.keys():
        if key == "-1" or key == -1:
            continue
        nb_of_class += 1
    for key, value in cluster_by_idx.items():
        if key == "-1" or key == -1:
            continue
        if len(set(value)) != 1:
            not_cross = False
            break

```

Figure 4.2. Clustering function

- Generate feature vectors of each packer:

```
def frequency_feature_vector_computation(packer_name, packed_files):
    print("Process packer: {}".format(packer_name))
    data = {}
    merged_unique_labels = []
    opcode_sequence = {}
    for packed_file in packed_files:
        _, file_name = get_packer_name_and_filename(packed_file)
        packed_dot_file = os.path.join(data_folder_path, "asm_cfg", packer_name,
                                       "{}_model.dot".format(packed_file))
        oep_address = oep_dictionary_2[packed_file]
        print("oep_address = {}".format(oep_address))
        preceding_oep, msg = get_preceding_oep(packed_dot_file, oep_address)
        if not preceding_oep:
            print("Packer: {}, file_name: {}, error: {}".format(packer_name, packed_file, msg))
            continue
        packed_graph = get_removed_backed_graph(packer_name, file_name)
        node_list_packed_file, node_labels_packed_file, original_labels_packed_file, _ = convert_graph_to_vector(
            packed_graph,
            address=preceding_oep)

        data[file_name] = Counter(list(node_labels_packed_file.values()) + original_labels_packed_file)
        opcode_sequence[file_name] = get_opcode_sequence(packed_graph, preceding_oep)
        merged_unique_labels = sorted(
            list(set(merged_unique_labels + list(node_labels_packed_file.values()) + original_labels_packed_file)))

    return data, opcode_sequence, merged_unique_labels
```

Figure 4.3. Calculate feature vectors' frequencies

> common	1	{
v configs	2	"0": {
v standard_feature_vectors	3	"1-GETMODULEHANDLE-KERNEL32-DLLcall": 1.0,
{} aspack.json	4	"1-GETPROCADDRESS-KERNEL32-DLLcall": 0.2857142857142857,
{} fsg.json	5	"1-VIRTUALALLOC-KERNEL32-DLLcall": 0.2857142857142857,
{} jdpack.json	6	"1-VIRTUALFREE-KERNEL32-DLLcall": 0.2857142857142857,
{} mew.json	7	"1-VIRTUALPROTECT-KERNEL32-DLLcall": 1.0,
{} MPRESS.json	8	"1-addladdl": 1.0,
{} packman.json	9	"1-addlandl": 0.2857142857142857,
{} pecompact.json	10	"1-addljne": 1.0,
{} petitepacked.json	11	"1-addlmovl": 1.0,
{} telock.json	12	"1-addlpopl": 0.2857142857142857,
{} upx.json	13	"1-addlpushl": 1.0,
{} winupack.json	14	"1-addlshrl": 0.2857142857142857,
{} yodaC.json	15	"1-addlsubl": 0.2857142857142857,
≡ end_of_unpacking_sequence_mixpa... M	16	"1-andlmovl": 0.2857142857142857,
≡ end_of_unpacking_sequence.txt M	17	"1-andlpushl": 0.2857142857142857,
	18	"1-andlsubl": 0.2857142857142857,
	19	"1-callcall": 0.2857142857142857,
	20	"1-calljmp": 1.0,

Figure 4.4. Feature vectors (label + frequency) of each packer

- Generating label for each group of packer and checking the consistency of end sequence within a group:

```

for idx in range(len(packed_files)):
    label = str(labels[idx])
    print("label = {}".format(label))
    if label == "-1":
        continue
    print("pass: {}".format(type(label)))
    if not label in opcode_sequence_of_label:
        opcode_sequence_of_label[label] = []

    _, file_name = get_packer_name_and_filename(packed_files[idx])
    opcode_sequence_of_label[label].append(opcode_sequence[file_name])

    if label in n_sample_of:
        n_sample_of[label] += 1
    else:
        n_sample_of[label] = 1

if not (label in standard_feature_vectors):
    standard_feature_vectors[label] = {}

for idy, label_of_node in enumerate(merged_unique_labels):
    if not (label_of_node in standard_feature_vectors[label]):
        standard_feature_vectors[label][label_of_node] = 0.0
        standard_feature_vectors[label][label_of_node] += X[idx][idy]
# Check condition opcode sequence in group have to have the same sequence > 0
for cluster_id, opcode_sequences in opcode_sequence_of_label.items():
    for idx in range(len(opcode_sequences)):
        if opcode_sequences[idx][:first_k] != opcode_sequences[0][:first_k]:
            return standard_feature_vectors, opcode_sequence_of_label, None

```

Figure 4.5. Function to form templates for each packer

- Saving end sequences of unpacking stubs:

```

# Save end-of-sequence
with open("configs/end_of_unpacking_sequence_mixpacker.txt", "a") as f:
    for label, opcode_sequences in optimal_opcode_sequence_of_label.items():
        print("label sequence = {}".format(label))
        for opcode_sequence in opcode_sequences:
            print(opcode_sequence)
        max_length = min([len(seq) for seq in opcode_sequences])
        is_diff = False
        for idx in range(0, max_length):
            opcode = opcode_sequences[0][idx]
            for opcode_sequence in opcode_sequences:
                if opcode_sequence[idx] != opcode:
                    is_diff = True
                    break
            if is_diff:
                line = "_".join(opcode_sequences[0][:idx])
                f.writelines("{}{}\n".format(packer_name, line))
                break
        if not is_diff:
            line = "_".join(opcode_sequences[0][:max_length])
            f.writelines("{}{}\n".format(packer_name, line))
    return optimal_standard_feature_vectors

```

Figure 4.6. Function to save end sequences of unpacking stubs

Finally, we got a Weisfeiler-Lehman histogram vector along with an end sequence for each cluster of packer as a template for it.

We do template matching by setting up the template for the input packed file, then calculate the cosin value between 2 templates (one of the input file with the available templates). Base on the best result of the comparison, we finally give final decision about (packer name, OEP address and the score of similarity).

```
def end_unpacking_prediction(packer_name, group_label, node_list, unique_labels, data, end_unpacking_sequences, first_k=-1):
    best_similarity = 0
    save_address = None
    try:
        histograms = {}
        print("Calculating histogram:")
        for idx, label in enumerate(unique_labels):
            for name in ['G1'] + node_list:
                if not (name in histograms):
                    histograms[name] = []
                if label in data[name]:
                    histograms[name].append(data[name][label])
                else:
                    histograms[name].append(0)
        print("Searching for best matching graph:")
        for name in node_list:
            check_end_unpacking_sequence = True if first_k == -1 else False
            for end_unpacking_seq_sample in end_unpacking_sequence_samples[packer_name][int(group_label):int(group_label)+1]:
                if end_unpacking_sequences[name][:first_k] == end_unpacking_seq_sample[:first_k]:
                    check_end_unpacking_sequence = True
            if not check_end_unpacking_sequence:
                continue
            sim = cosine_similarity_oe(histograms['G1'], histograms[name])
            if sim > best_similarity:
                best_similarity = sim
                save_address = name
    except Exception as e:
        return None, None, str(e)
    return save_address, best_similarity, "Success"
```

Figure 4.8. End of unpacking prediction

```

for packer_name_candidate, sample_files in tqdm(standard_feature.items()):
    for group_label, feature in sample_files.items():
        # update information of sample file
        data['G1'] = feature
        # create unique labels of G1 and sub graphs
        merged_unique_labels = sorted(
            list(set(unique_labels + list(feature.keys()))))
        # Finding end-of-unpacking
        predicted_address, score, msg = end_of_unpacking_prediction(packer_name=packer_name_candidate,
                                                                    group_label=group_label,
                                                                    node_list=node_list,
                                                                    unique_labels=merged_unique_labels,
                                                                    data=data,
                                                                    end_unpacking_sequences=end_unpacking_sequences,
                                                                    first_k=args.first_k)

        if score is None:
            print("Packer: {}, file_name: {}, error: {}".format(
                packer_name, file_name, msg))
            log_file.writelines(
                "Packer: {}, file_name: {}, error: {}\n".format(packer_name, file_name, msg))
            continue
        if score > final_score:
            final_score = score
            final_address = predicted_address
            predicted_packer = packer_name_candidate

```

Figure 4.9. Final decision for packer identification and OEP detection

4.1.4. IAT Rebuild Module

We are now building the dumping module and IAT rebuilding plus sections fixing one to serve for the process of unpacking, especially recovering the original executable payload. However, at present, this module is just working with cases of packed PE files using several certain API obfuscation techniques. We still need to research and test more cases to improve the effectiveness of the used algorithm.

In short, our method to build this module is now mainly based on the following principles:

- **Dumping the OEP memory** (can obtain at the end of unpacking stub) of the packed file. However, a typical approach for generic unpacking tools is to run the packed program, allowing the unpacking stub to complete its task. Once this is done, the original payload can be dumped from memory, and any required modifications to the PE Header can be applied.

- **Unmapping PE files**

The purpose of unmapping a File is to make the sections line up correctly. In the mapped PE, sections are not lined up correctly due to this we cannot see informations like imports and exports of the PE. By unmapping we can make the sections line up correctly and view information like imports and exports of the portable executable file.

Steps:

1. Access the Section Hdrs information.

Disasm	General	DOS Hdr	Rich Hdr	File Hdr	Optional Hdr	Section Hdrs	BaseReloc.	
+ [icon]								
Name	Raw Addr.	Raw size	Virtual Addr.	Virtual Size	Characteristics	Ptr to Reloc.	Num. of Reloc.	Num. of Linenum.
> .text	400	21C00	1000	3471000	60000020	0	0	0
> .rdata	22000	200	23000	3493000	40000040	0	0	0
> .data	22200	400	24000	3494000	C0000040	0	0	0
> .reloc	22600	400	26000	3496000	42000040	0	0	0

Figure 4.10. Section Hdrs information

2. Make the Raw address match the Virtual address of Sections.

3. Set the Raw size and Virtual size correctly using this formula

Raw size of Section n = VA of section $(n + 1)$ – VA of section n .

Ex: *Raw size of first Section (.text 22000) = Virtual Address of Section 2 (.rdata = 23000) – Virtual Address of Section 1 (.text = 1000).*

4. Follow this formula for all Other section except reloc.

5. Make the Virtual Size match the Raw size for all sections except reloc.

6. Make reloc size 0 because reloc section only exist in disk. Since we have dumped the pe from memory it will not have reloc section.

7. Make sure the image base is same as the packed binary's image base.

- Fixing alignment and fixing corrupted/missing PE header

Some malware authors corrupt the PE headers of unpacked sample. So after dumping the memory we need to add PE header to the file.

Steps:

1. First add the correct PE header: Look for 4C 01 / 64 86 (CPU architecture in optional header) open the corrupted file and a known good PE file and copy and paste the PE header of good file (till 4C 01 / 64 86) to the corrupted file (till 4C 01 / 64 86)

2. Check and fix section alignment: Move to raw address of Section 1. Zero out bytes till the start of section 1. If there are any non-zero bytes, insert null bytes (00) to fill the gap.

- *Finding all import APIs and fixing the IAT*

We currently apply this algorithm to reconstruct the IAT with the input of the process ID, OEP offset, dumped section data that contains OEP from the previous step:

First, bypass several API obfuscation techniques to scan for all IAT pointer candidates in the specified address space:

To address direct call and unconditional jumps, the algorithm checks for direct calls and jumps (CALL or unconditional JMP) to absolute memory addresses. If such an address is found, it will extract the first argument and masked it to a 32-bit address, and assigned to a variable for later use.

Towards handling indirect calls through registers, we follow this logic: When the algorithm detects a CALL through a register, it attempts to resolve the register's value using a defined dictionary. This dictionary stores known values of registers that were previously assigned in the code flow. Similar logic applies for JMP through registers. We will check if the register used for the jump is mapped to a known memory address and retrieves it. To support this action, we need to track register values after each instruction. For MOV instructions that move an absolute memory address into a register, we will update its value in the aforementioned dictionary. This ensures that indirect control flow instructions that rely on register contents can be resolved later.

Handling PUSH-RET sequences: Many packers avoid CALL and JMP instructions, instead, using a pair of instructions PUSH-RET such as: Aspack, UPX, FSG, PECompact, Telock, Themida,... When encountering a PUSH to an absolute memory address, the address is pushed onto a stack. This is a common pattern used in return-oriented programming to manipulate control flow. On encountering a RET instruction, if there are values on the stack, it pops an address

from the saved specified stack and assigns it as an IAT pointer. This reflects control flow redirection through PUSH-RET sequences.

Dealing with Structured Exception Handling (SEH) redirection: Firstly, we need to check for common SEH patterns such as FS segment usage, which is typical for SEH registration. You can analyze MOV instructions that reference the FS register (e.g., MOV EAX, FS:[0]). Also, track instructions that attempt to trigger exceptions, such as division by zero (DIV) or access violations. Then we can choose to dynamically emulate these trampolines and identify if they eventually land in legitimate API addresses by resolving the final destination after SEH redirection; or we can simply trace the disassemble code of our program, spot any memory access involving the FS register, because of the fact that most SEH redirection typically occurs when the program uses the FS segment register to handle exceptions; then continue our algorithm to scan for IAT pointer in this new SEH direction. In this case, we apply the latter one.

For stolen code techniques, we need to identify control flow redirection that starts by copying a few bytes from an API function and then returning to the API after these copied bytes. This usually involves indirect calls or jumps, where control flow is manipulated via trampolines or other mechanisms. Our algorithm is now try to manage indirect CALL and JMP, then if the address stored in the register is not a valid API address, we will scan in the range ± 10 to find out the valid one.

Other techniques of API obfuscation mentioned in section 1.4.2, we are now finding an effective algorithm to manage.

- Checking and dealing with remained obfuscation or malfunctioning-cause techniques

This is one of the most challenging stages in the process of recovering executable payloads and is frequently responsible for failed recovery attempts. There are often residual obfuscation techniques, such as encrypted sections, anti-debugging mechanisms, or tampered control flow, that can still hinder the

accurate restoration of the original executable. These techniques can cause instability, crashes, or prevent the payload from functioning as expected.

Currently, this process is largely manual, relying heavily on the expertise of experienced reverse engineers who are able to detect subtle signs of obfuscation and apply workarounds. However, advancements in automated analysis tools are gradually helping to reduce the manual effort needed, by detecting common obfuscation patterns or simulating runtime environments to resolve execution issues. Still, human intervention remains critical for dealing with complex, custom obfuscation techniques that have not yet been automated. Integrating dynamic analysis, runtime monitoring, and symbolic execution techniques can also provide additional insights into resolving such obfuscation challenges.

4.1.5. Diagram of the Solution

After taking a thorough look at each step of generating CFGs, detecting packer name and OEP, dumping and fixing IAT to obtain the original executable payload, we combine them into a comprehensive diagram depicting the whole process with 3 separated modules:

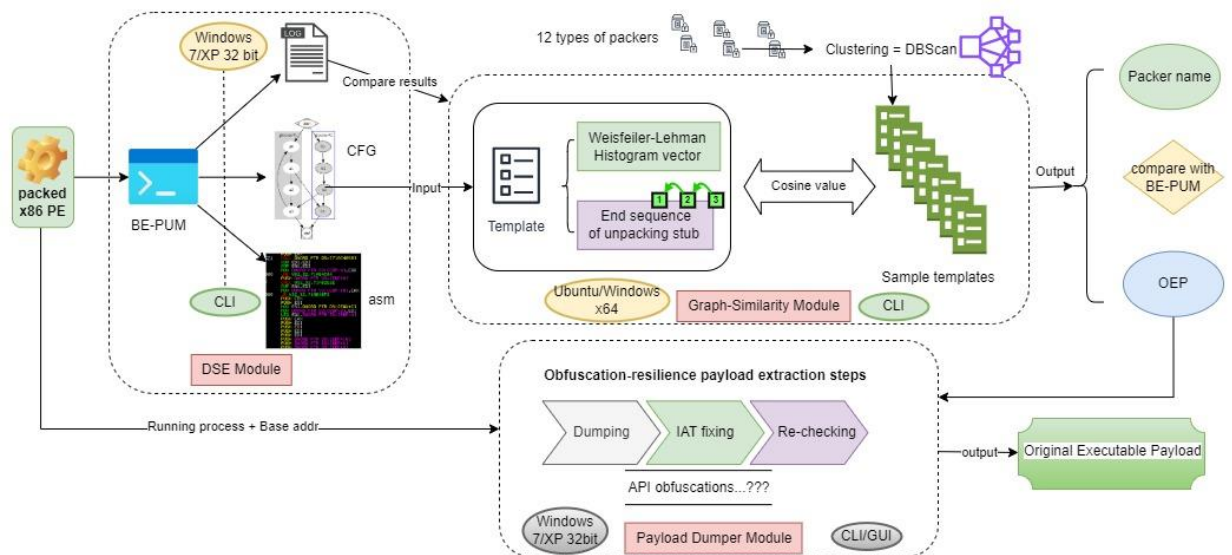


Figure 4.11. Diagram of the solution supporting reversing packed files

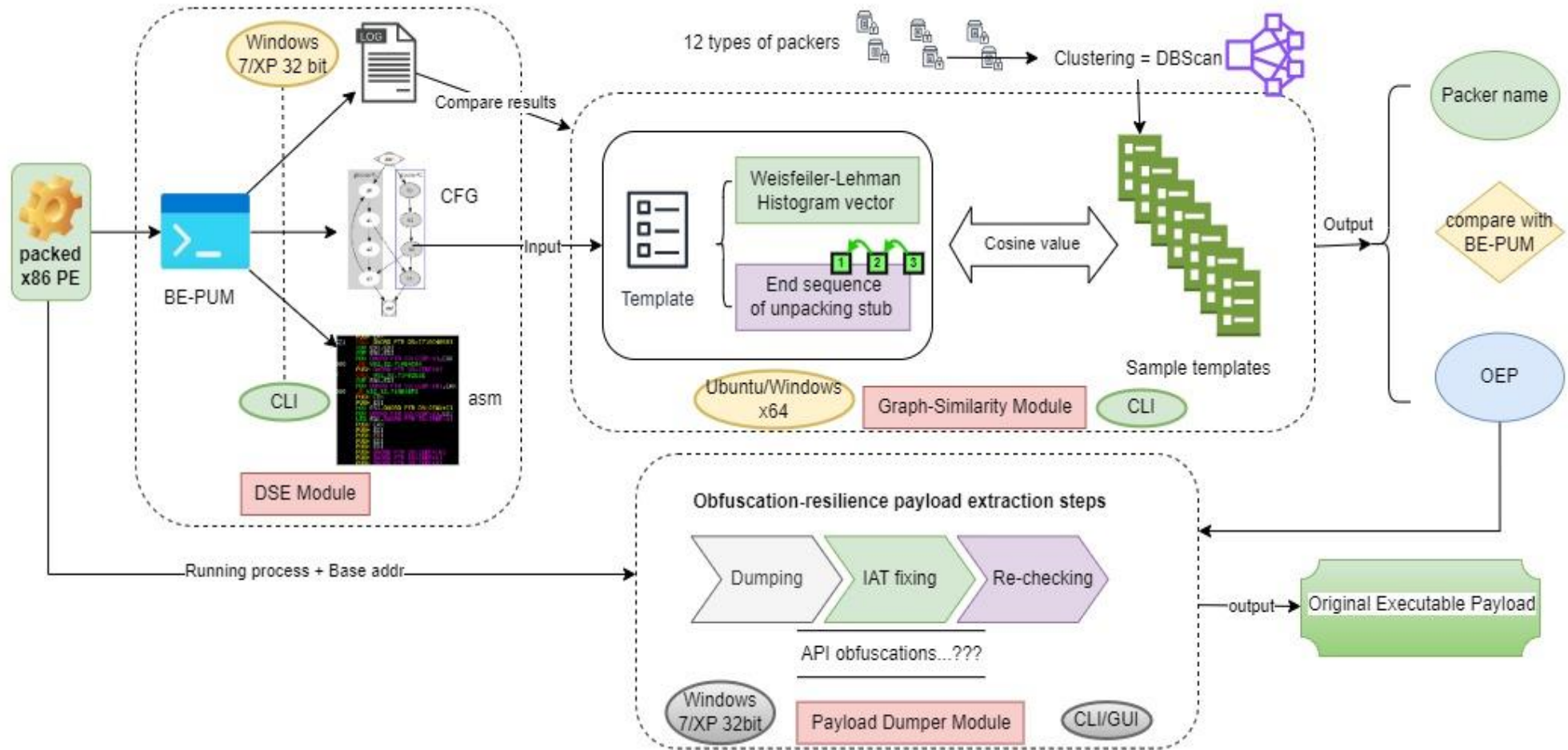


Figure 4.12. Diagram of the solution supporting reversing packed files (zoomed-in image)

4.2. Experiments

4.2.1. Packer Identification and CFG Generation Using BE-PUM

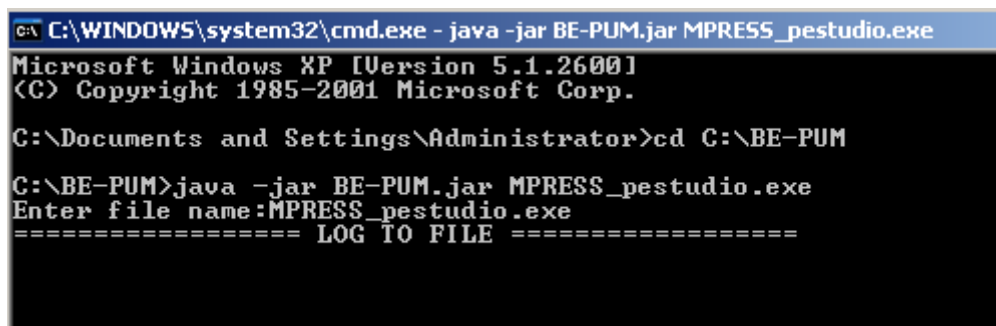
- **Environment:**

- Windows XP 32 bit
- BE-PUM project
- Installed requirements

- **Demo 1:**

Input: *MPRESS_pestudio.exe* (packed by MPRESS)

Execution:



```
C:\WINDOWS\system32\cmd.exe - java -jar BE-PUM.jar MPRESS_pestudio.exe
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

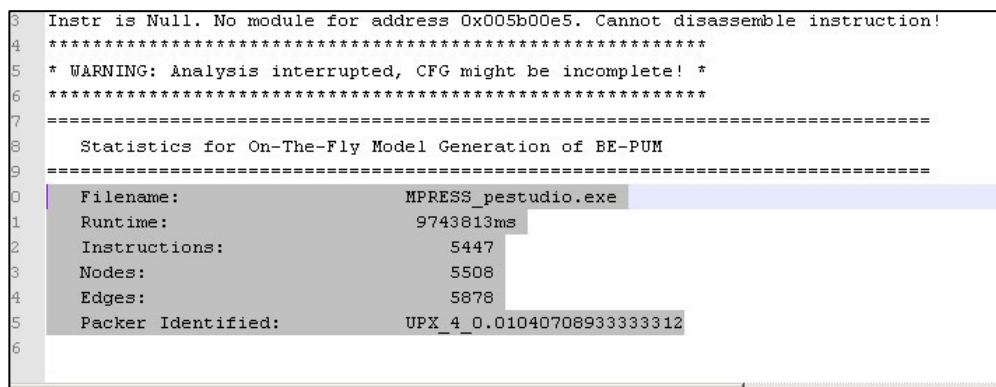
C:\Documents and Settings\Administrator>cd C:\BE-PUM

C:\BE-PUM>java -jar BE-PUM.jar MPRESS_pestudio.exe
Enter file name:MPRESS_pestudio.exe
===== LOG TO FILE =====
```

Figure 4.13. Execution of BE-PUM in test 1

Output:

- *Log-MPRESS_pestudio.exe.log*



```
3 Instr is Null. No module for address 0x005b00e5. Cannot disassemble instruction!
4 *****
5 * WARNING: Analysis interrupted, CFG might be incomplete! *
6 *****
7 =====
8 Statistics for On-The-Fly Model Generation of BE-PUM
9 =====
10 Filename: MPRESS_pestudio.exe
11 Runtime: 9743813ms
12 Instructions: 5447
13 Nodes: 5508
14 Edges: 5878
15 Packer Identified: UPX_4_0.01040708933333312
16
```

Figure 4.14. Log file of BE-PUM test 1

- *MPRESS_pestudio.exe_model.dot*
- *MPRESS_pestudio.exe_code.asm*

Accuracy of packer identification: **False** (This case, BE-PUM mistakenly identifies MPRESS as UPX, which is a common issue during the analysis)

Result:

```

Packer: pcompact, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 27.000, packer_
r_identification accuracy: 27.0, be-pum: 27.0, of sample: 27
Packer: mew, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 75.000, packer_iden
tification accuracy: 75.0, be-pum: 75.0, of sample: 75
Packer: petitepacked, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 34.000, pa
cker_identification accuracy: 34.0, be-pum: 34.0, of sample: 34
Packer: upx, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 85.000, packer_iden
tification accuracy: 85.0, be-pum: 84.0, of sample: 85
Packer: aspack, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 68.000, packer_i
dentification accuracy: 68.0, be-pum: 68.0, of sample: 68
Packer: winupack, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 26.000, packer
_identification accuracy: 26.0, be-pum: 26.0, of sample: 26
Packer: packman, end-of-unpacking accuracy: 0.987, OEP detection accuracy: 78.000, packer_
identification accuracy: 79.0, be-pum: 0.0, of sample: 79
Packer: telock, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 27.000, packer_i
dentification accuracy: 27.0, be-pum: 0.0, of sample: 27
Packer: MPRESS, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 78.000, packer_i
dentification accuracy: 78.0, be-pum: 78.0, of sample: 78
Packer: jdpack, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 52.000, packer_i
dentification accuracy: 52.0, be-pum: 0.0, of sample: 52
Packer: fsg, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 75.000, packer_iden
tification accuracy: 75.0, be-pum: 75.0, of sample: 75
Packer: yodaC, end-of-unpacking accuracy: 1.000, OEP detection accuracy: 74.000, packer_id
entification accuracy: 74.0, be-pum: 62.0, of sample: 74

```

Figure 4.17. Results of packer and OEP detection of BE-PUM and graph-based method

Initially, when re-applying the method with the dataset, there was a problem with the result of packer detection for WINUPACK: The algorithm has failed with half of packed PE files which are packed using WINUPACK.

The reason behind this phenomenon we finally found out is that each packer can have more than one template. We spot another template of WINUPACK generated by WINUPACK version Alt stub.

```

}
else if(PE.compareEP("BE.....AD50FF7634EB7C4801"))
{
    switch(PE.readByte(PE.nEP+62))
    {
        case 0x37: sVersion="0.37 beta"; break;
        case 0x38: sVersion="0.38 beta"; break;
        case 0x39: sVersion="0.39 final"; break;
        case 0x3A: sVersion="0.399"; break;
    }
    bDetected=1;
}

```

Figure 4.18. Several versions of Winupack handled by DIE


```

}
else if(PE.compareEP("60E809000000.....33C95E870E"))
{
    sOptions="Alt stub";
    bDetected=1;
}

```

Figure 4.19. Two versions of WINUPACK v.039f / Alt stub detected by DIE

To address this issue, we collect more samples packed by WINUPACK and establish new templates for WINUPACK. The result obtained from this modification does not disappoint us with the successful rate of 55/55 test samples.

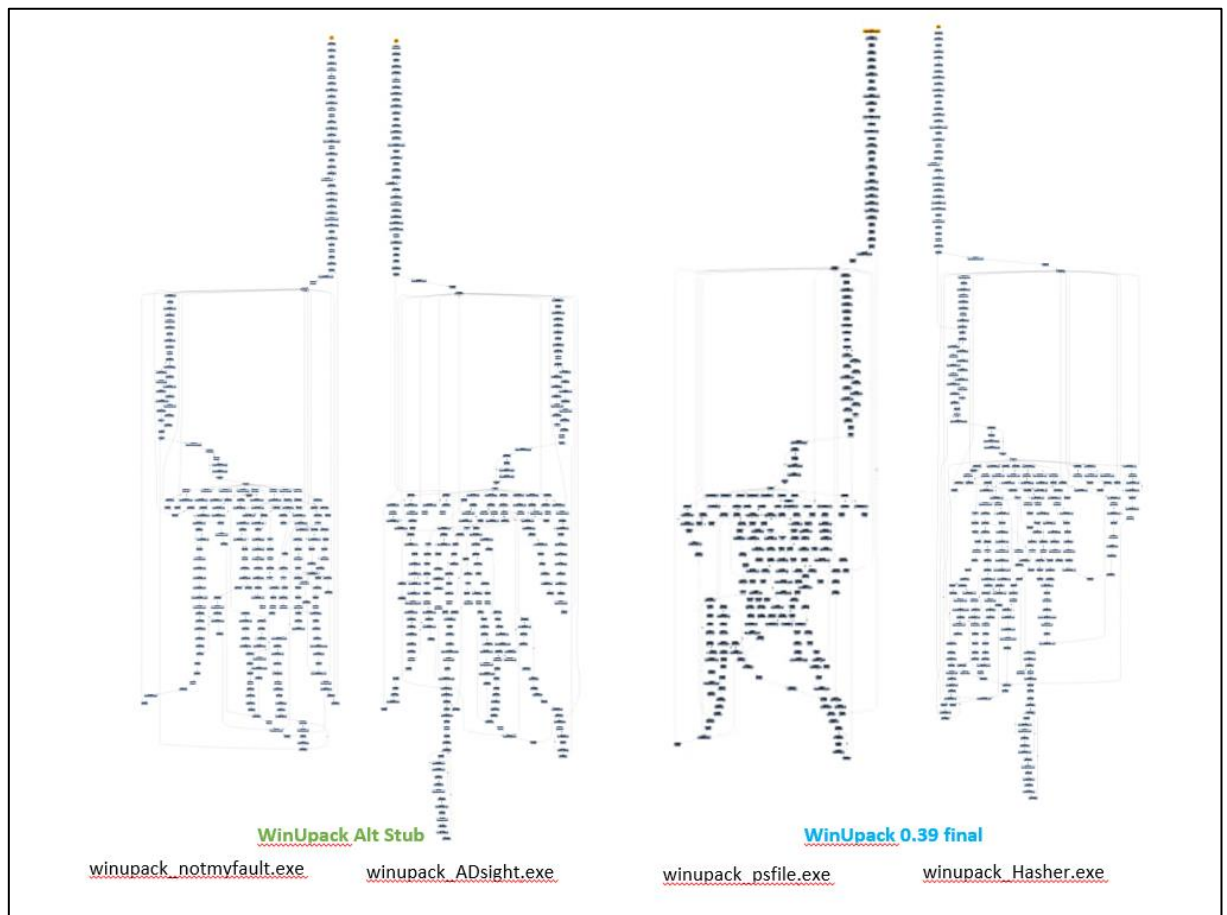


Figure 4.20. Two special templates of WinUpack unpacking stubs

We also collect about 300 samples x86 malwares in <https://bazaar.abuse.ch/browse.php?search=tag%3Ax86> and test packer detection. Most of the malware samples collected are none-packed files, however, there are a number of samples identified as having been packed with npack

packer. Therefore, we utilized those samples to set up templates for the 13th packer: npack.

Demo with a packed file:

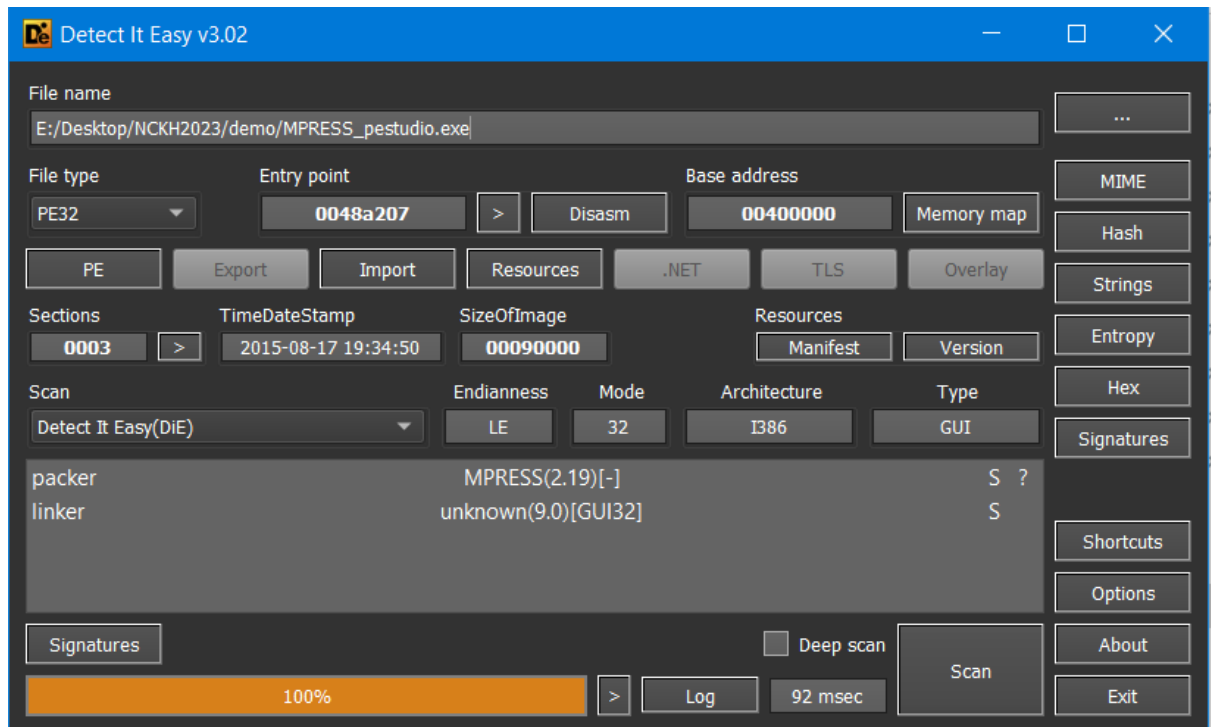


Figure 4.21. Information of the input packed file

```
python3 MTA_JAIST_x86unpack.py --logbp '/home/william/Desktop/Log-
MPRESS_pestudio.exe.log'
--dotbp
'/home/william/Desktop/MPRESS_pestudio.exe_model.dot' --file pestudio.exe
```

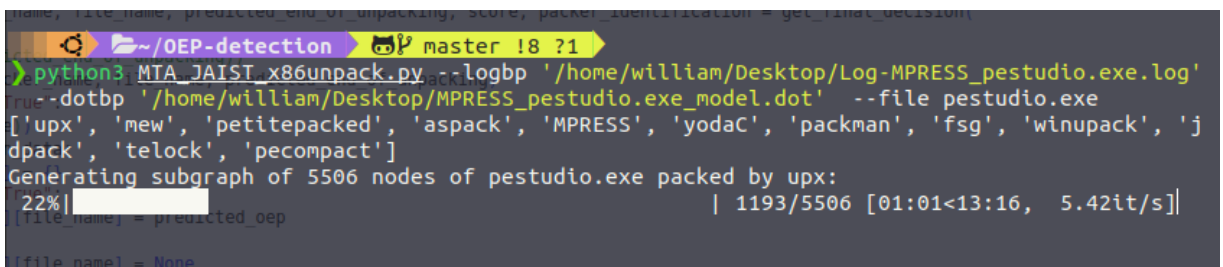


Figure 4.22. Demo with MPRESS_pestudio.exe file

Result: Graph-based method recognize the right packer (which is missed by BE-PUM in the previous test) and precise OEP:



Figure 4.23. Result of graph-based method and BE-PUM

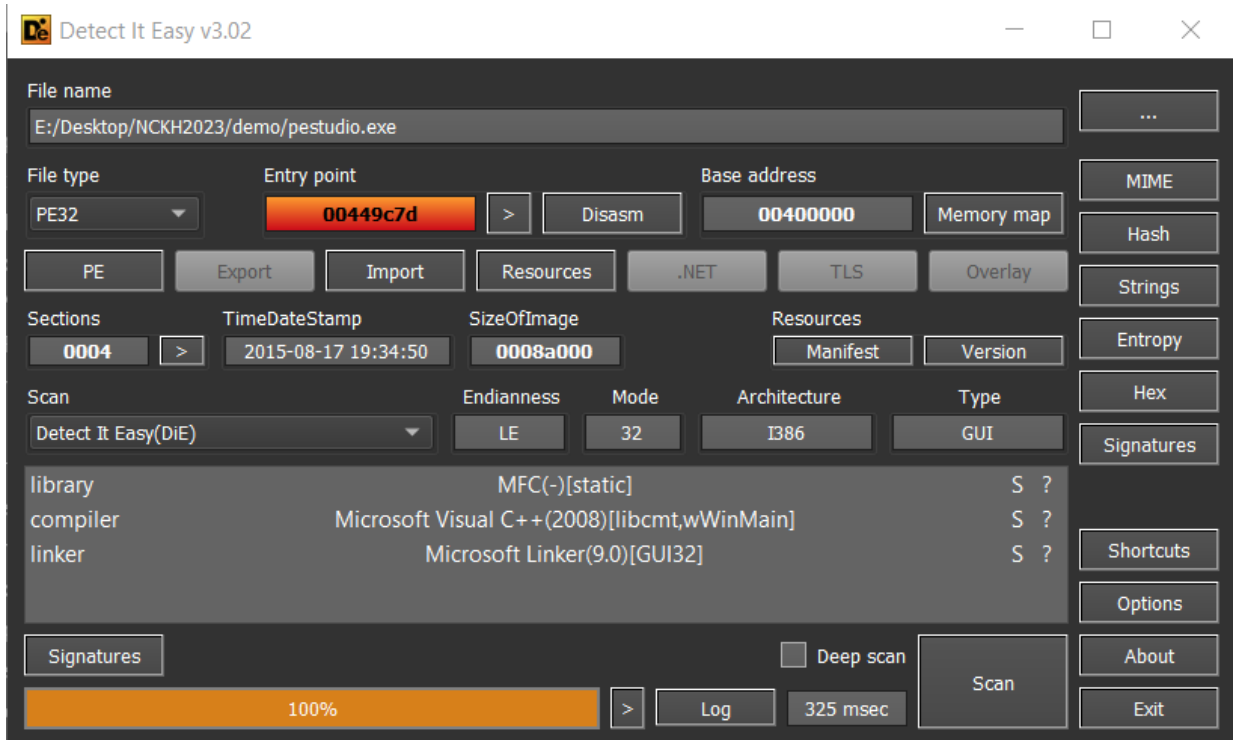


Figure 4.24. Compare with the original file

4.2.3. Extracting Original Payloads Manually

To obtain the original executable payloads manually, we will choose x32dbg and Scylla plugin. And below is the detailed process of dumping:

Input: aspack_ADInsight.exe (OEP: 0x00434DAC)

First, we debug the file and use PUSHAD method to find OEP:

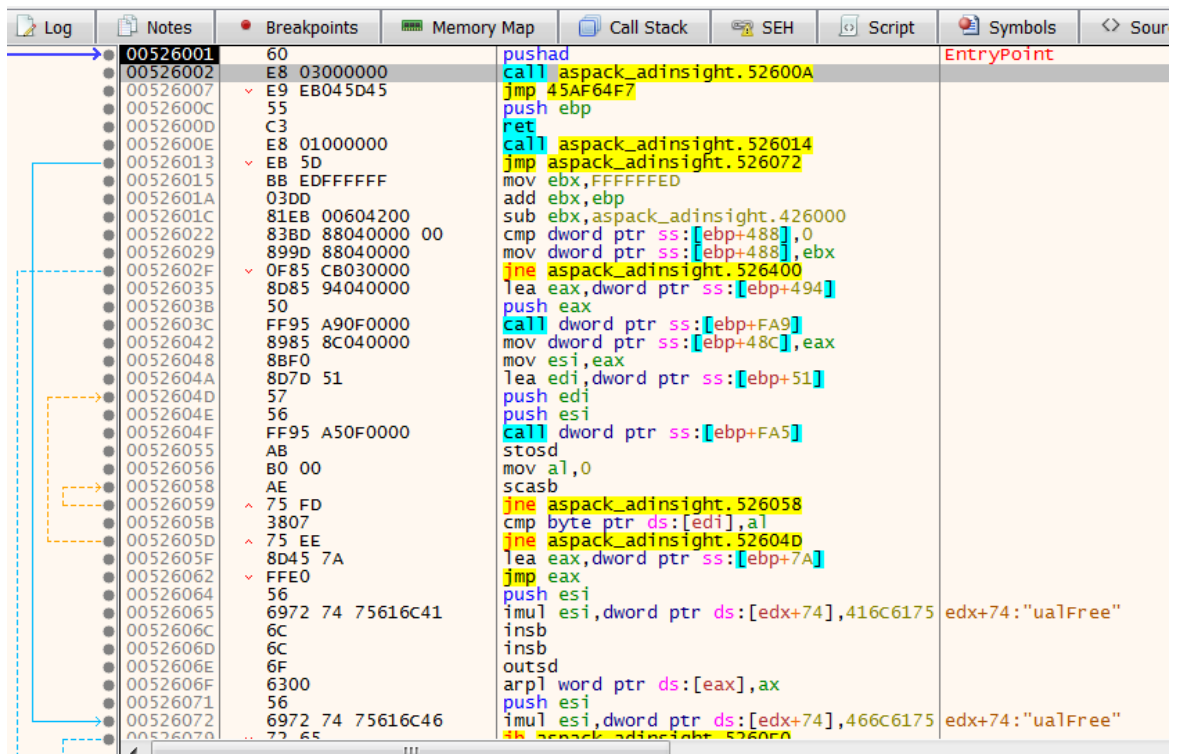


Figure 4.25. Load file into x32dbg

After continuing to run and stopping at the breakpoint, we successfully encounter the well-known PUSH-RET sequence, which is commonly seen in packed executables. This sequence often signals a transition point, leading us closer to identifying the OEP, where the unpacked code begins executing. By locating this point, we can prepare to dump the executable and further analyze its unpacked state.

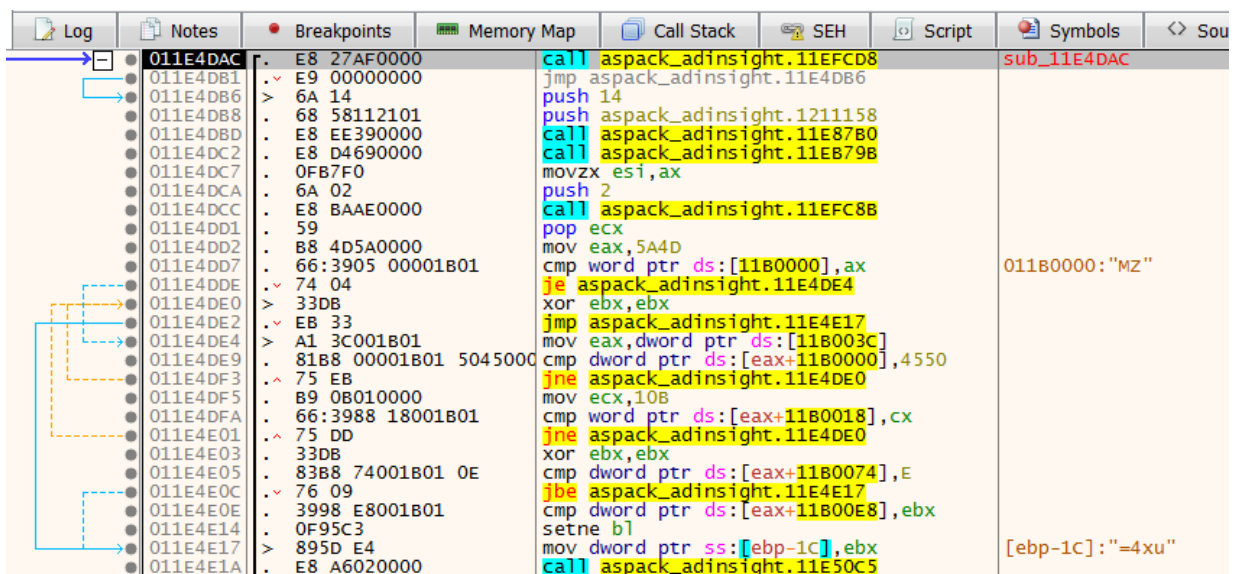


Figure 4.26. Locate to the OEP

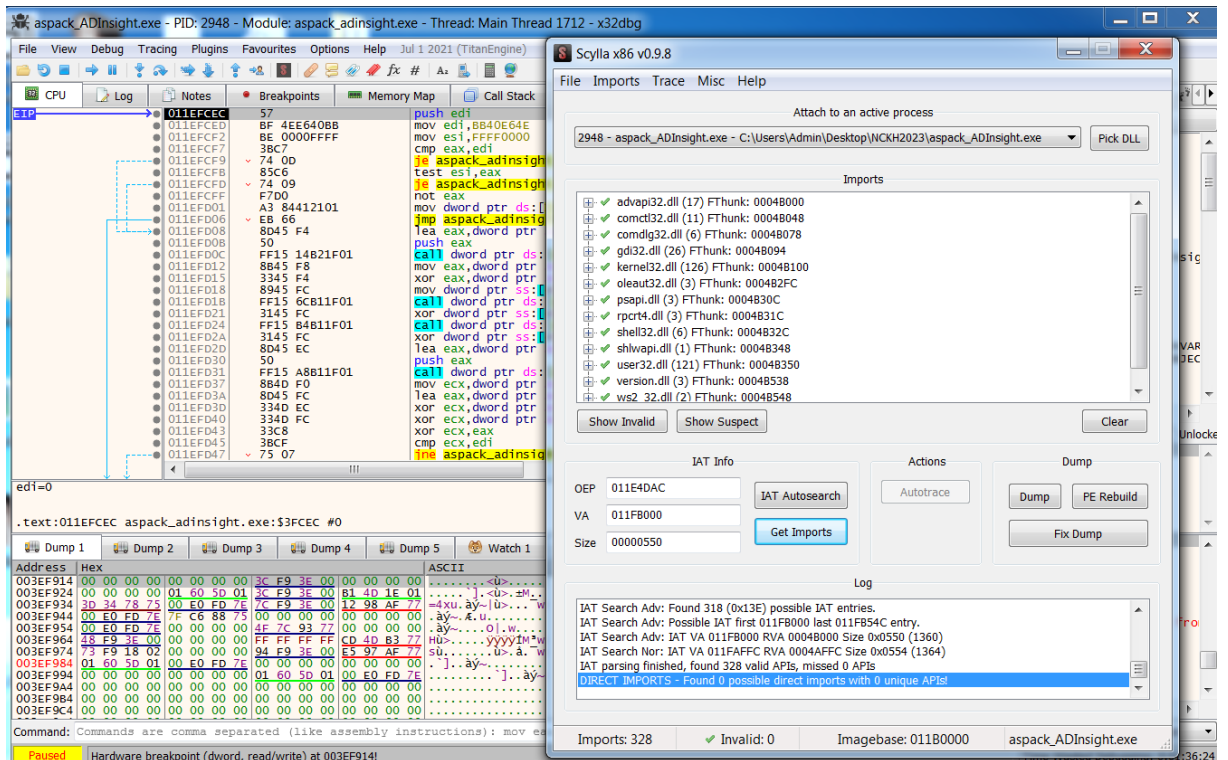


Figure 4.27. Get imports successfully

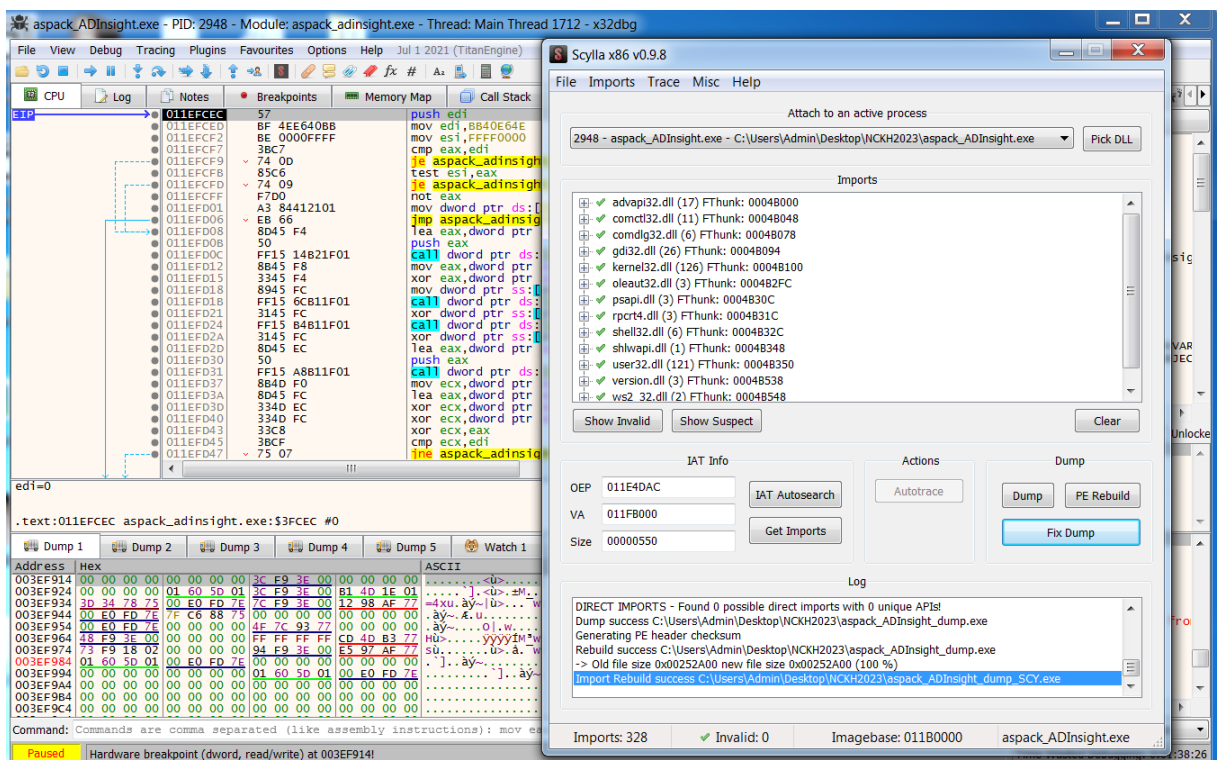


Figure 4.28. Dump, fix dump and rebuild IAT with Scylla

ADInsight.exe							aspack_ADInsight_dump_SCY.exe						
Module Name	Imports	OFFs	TimeStamp	ForwarderChain	Name RVA	FTs (IAT)	Module Name	Imports	OFFs	TimeStamp	ForwarderChain	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword	szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
COMCTL32.dll	11	000618E8	00000000	00000000	00061ECC	0004B048	advapi32.dll	17	0042C000	00000000	00000000	0042C66C	0004B000
WS2_32.dll	2	00061DE8	00000000	00000000	00061EDA	0004B548	comctl32.dll	11	0042C048	00000000	00000000	0042C7C9	0004B048
RPCRT4.dll	3	00061BBC	00000000	00000000	00061F16	0004B31C	comdlg32.dll	6	0042C078	00000000	00000000	0042C8BE	0004B078
PSAPI.dll	3	00061BAC	00000000	00000000	00061F5E	0004B30C	gd32.dll	26	0042C094	00000000	00000000	0042C926	0004B094
VERSION.dll	3	00061DD8	00000000	00000000	00061FAA	0004B538	kernel32.dll	126	0042C100	00000000	00000000	0042CAB2	0004B100
KERNEL32.dll	126	000619A0	00000000	00000000	0006258A	0004B100	oleaut32.dll	3	0042C2FC	00000000	00000000	0042D380	0004B2FC
USER32.dll	121	00061BF0	00000000	00000000	00062078	0004B350	psapi.dll	3	0042C30C	00000000	00000000	0042D3BD	0004B30C
GDI32.dll	26	00061934	00000000	00000000	00062F16	0004B094	rpcrt4.dll	3	0042C31C	00000000	00000000	0042D401	0004B31C
COMDLG32.dll	6	00061918	00000000	00000000	00062F7E	0004B078	shell32.dll	6	0042C32C	00000000	00000000	0042D43A	0004B32C
ADVAPI32.dll	17	000618A0	00000000	00000000	000630E4	0004B000	shlwapi.dll	1	0042C348	00000000	00000000	0042D4B5	0004B348
SHELL32.dll	6	00061BCC	00000000	00000000	00063164	0004B32C	user32.dll	121	0042C350	00000000	00000000	0042D4D0	0004B350
OLEAUT32.dll	3	00061B9C	00000000	00000000	00063170	0004B2FC	version.dll	3	0042C538	00000000	00000000	0042DC81	0004B538
SHLWAPI.dll	1	00061BE8	00000000	00000000	0006318E	0004B348	ws2_32.dll	2	0042C548	00000000	00000000	0042DCCF	0004B548

Figure 4.29. Comparison between two IATs

We noticed that the newly reconstructed IAT is filled with the same APIs from the same DLLs when compared to the original one; however, the dumped file still can not run normally.

There are several reasons why a dumped program may have a correctly reconstructed IAT but still fail to run:

Many packers include anti-debugging or anti-tampering techniques that may disrupt the execution of the dumped executable. Even if the IAT is properly restored, these mechanisms may detect alterations and cause the program to malfunction or terminate.

Another reason to be accused of is that during the dumping process, sections of the PE file might be misaligned or improperly rebuilt. Even with a valid IAT, if the program's sections (such as code, data, or resource sections) are not restored correctly, the binary may fail.

In this case, we found out that scylla did not turn off the option "DLL can move" when dumping. If the "DLL can move" option is enabled in the DLL characteristics, the operating system may load the DLL at a different base address than its preferred one. This is called rebasing. In the case of a dump file, if the file relies on absolute memory addresses and isn't correctly adjusted to account for relocation, enabling this option could lead to crashes or malfunctions when the program attempts to access incorrect memory addresses that have not been properly relocated after the move.

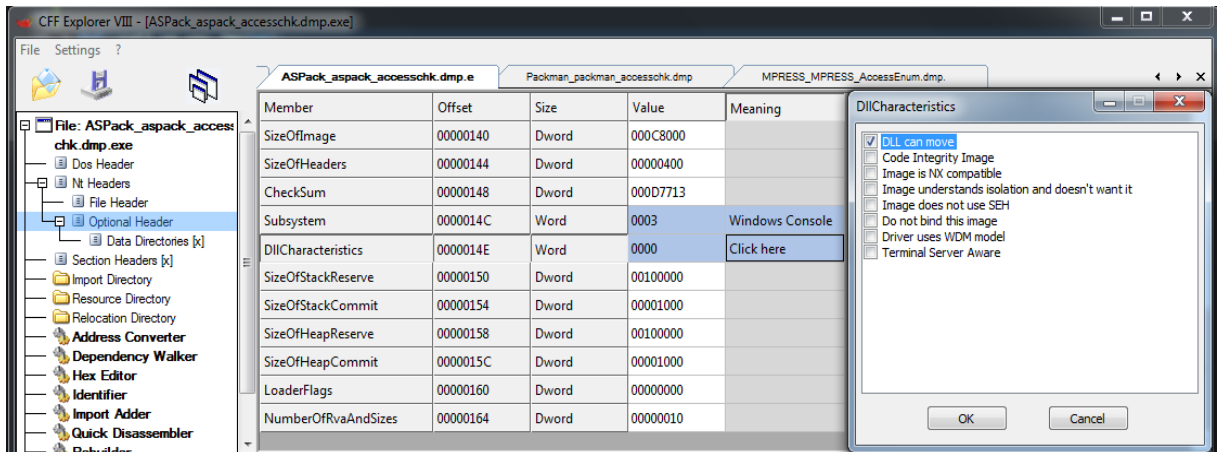


Figure 4.30. DLL characters shown in CFF Explorer

4.2.4. Extracting Original Payloads Using PE Dump and IAT Rebuild Module

Data preparation: First, we set up the environment to run x86 PE files (Vmware virtual machine Windows 7 32-bit version). Then we run a full functionality test helping us filter out 437 files from a total of 729 files that were used in the previous section.

During the dumping and IAT fixing stage, the module successfully processed and dumped 318 out of the 437 files, accounting for approximately 72.77%. This step involved extracting the executable from memory and resolving the Import Address Table (IAT) to ensure the dumped files could be easily and precisely analysed.

We also conduct a similar test using *pyiatrebuild* and get the result of 77 over 437 files, accounting for over 17.62%.

Upon reviewing the code, it is clear that the *pyiatrebuild* tool primarily focuses on scanning for IAT pointers through direct calls, jumps, and indirect calls. However, it does not yet handle more sophisticated API obfuscation techniques that are often used in modern malware. These techniques, such as stolen bytes, dynamic API resolution, or opaque predicates, allow malware to hide or manipulate API calls in ways that evade simpler IAT reconstruction tools.

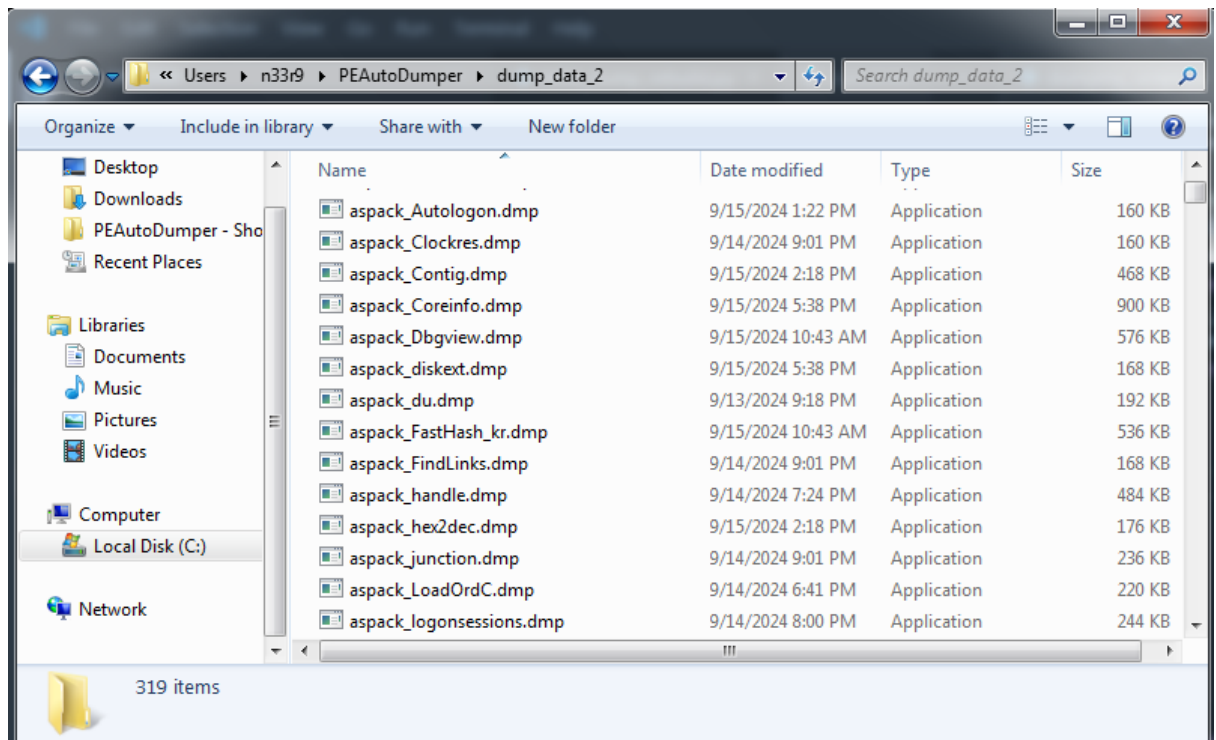


Figure 4.31. Result of dumping an fixing IATs module

```
def process_file(file_path, packer_dir, dump_folder_path):
    global should_stop
    if should_stop:
        return
    if is_pe(file_path):
        process = None
        try:
            log_message = "Processing file: %s" % file_path
            print(log_message)
            logger.info(log_message)
            dump_file_path = oss.path.join(dump_folder_path, oss.path.basename(file_path).split('.')[0] + '.dmp.exe')
            oep_offset = get_OEP_data(file_path, packer_dir)
            process = subprocess.Popen([file_path], shell=False)
            pid = process.pid
            if oep_offset is not None and pid is not None:
                pe_dump_data = dump_and_rebuild_script_auto(pid, oep_offset)
                with open(dump_file_path, 'wb') as f:
                    f.write(pe_dump_data)
                logger.info("Successfully processing file: %s" % file_path)
        except Exception as e:
            error_message = "Failed to process %s: %s" % (file_path, str(e))
            print(error_message)
            logger.error(error_message)
        finally:
            if process:
                try:
                    parent = psutil.Process(process.pid)
                    for child in parent.children(recursive=True):
                        child.terminate()
                    parent.terminate()
                    parent.wait(timeout=5)
                except psutil.NoSuchProcess:
                    pass
                except psutil.TimeoutExpired:
                    parent.kill()
```

Figure 4.32. Function to process dumping and rebuilding IATs

- Dumping and reconstruct the IAT using CLI app:
 - Input: PE file path, OEP offset (RVA)

- Output: Dumped PE file with a reconstructed IAT

```
python MTA_dump_rebuild.py dr --file pefile_data\ASPack\aspack_accesschk.exe --oep 50217
```

```
C:\Users\n33r9\PEAutoDumper (MTA_dump_rebuild_CLIapp)
(venv_pyiat) λ python MTA_dump_rebuild.py dump_rebuild --file C:\Users\n33r9\PEAutoDumper\real_PE_data\ASPack\ASPack_aspack_accesschk.exe --oep 50217
Processing file: C:\Users\n33r9\PEAutoDumper\real_PE_data\ASPack\ASPack_aspack_accesschk.exe
2024-09-18 17:28:09,088 - INFO - Processing file: C:\Users\n33r9\PEAutoDumper\real_PE_data\ASPack\ASPack_aspack_accesschk.exe
2024-09-18 17:28:09,088 - INFO - Start dumping the original payload

Accesschk v6.12 - Reports effective permissions for securable objects
Copyright (C) 2006-2017 Mark Russinovich
Sysinternals - www.sysinternals.com

WARNING: warning null reldesc
2024-09-18 17:28:09,227 - WARNING - warning null reldesc
2024-09-18 17:28:09,275 - INFO - Starting IAT rebuild for PID 4672
WARNING: warning null reldesc
2024-09-18 17:28:09,290 - WARNING - warning null reldesc
OEP: 0x40c429
Image Base: 0x400000
RVA OEP: 0xc429
2024-09-18 17:28:11,085 - INFO - Successfully processing file: C:\Users\n33r9\PEAutoDumper\real_PE_data\ASPack\ASPack_aspack_accesschk.exe
```

Figure 4.33. Successfully dumping payload

- Compare the dumped file obtaining from the previous step:

accesschk.exe		aspack_accesschk.exe				
Module Name	Imports	OFTs	TimeStamp	ForwarderChain	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
NETAPI32.dll	4	0002ECF4	00000000	00000000	0002EDA6	0001E25C
VERSION.dll	3	0002ED30	00000000	00000000	0002EDF6	0001E298
KERNEL32.dll	84	0002EBA0	00000000	00000000	0002F042	0001E108
USER32.dll	9	0002ED08	00000000	00000000	0002F0E2	0001E270
GDI32.dll	6	0002EB84	00000000	00000000	0002F138	0001E0EC
COMDLG32.dll	1	0002EB7C	00000000	00000000	0002F14E	0001E0E4
ADVAPI32.dll	56	0002EA98	00000000	00000000	0002F654	0001E000
wevtapi.dll	5	0002ED40	00000000	00000000	0002F6D0	0001E2A8

Figure 4.34. Initial IAT

The overall structure and organization of the IAT remain intact, with the corresponding functions and their references correctly mapped. However, there are slight changes in the order and inclusion of specific DLLs, which might be identified and added during unpacking process or a false positive result.

aspack_accesschk.dmp.exe						
Module Name	Imports	OFTs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
comdlg32.dll	1	00000000	00000000	00000000	000C60F0	000C5000
version.dll	3	00000000	00000000	00000000	000C6109	000C5008
srvcli.dll	2	00000000	00000000	00000000	000C6156	000C5018
gdi32.dll	6	00000000	00000000	00000000	000C6182	000C5024
wevtapi.dll	5	00000000	00000000	00000000	000C61D4	000C5040
kernel32.dll	76	00000000	00000000	00000000	000C624A	000C5058
netutils.dll	1	00000000	00000000	00000000	000C6797	000C518C
samcli.dll	1	00000000	00000000	00000000	000C67B7	000C5194
advapi32.dll	56	00000000	00000000	00000000	000C67DA	000C519C
ntdll.dll	8	00000000	00000000	00000000	000C6CC4	000C5280
user32.dll	9	00000000	00000000	00000000	000C6D77	000C52A4

Figure 4.35. Reconstructed IAT

Looking at the result, we noticed about 8 imported functions of ntdll.dll. All of these functions are called from kernel32.dll to ntdll.dll, so the total APIs from kernel32.dll is $76 + 8 = 84$ (which equals to the initial number).

ntdll.dll	8	00000000	00000000	00000000	000C6CC4	000C5280
user32.dll	9	00000000	00000000	00000000	000C6D77	000C52A4

OFTs	FTs (IAT)	Hint	Name
Dword	Dword	Word	szAnsi
N/A	000C6CCE	0000	RtlReAllocateHeap
N/A	000C6CE2	0000	RtlDeleteCriticalSection
N/A	000C6CFD	0000	RtlAllocateHeap
N/A	000C6D0F	0000	RtlSizeHeap
N/A	000C6D1D	0000	RtlEncodePointer
N/A	000C6D30	0000	RtlDecodePointer
N/A	000C6D43	0000	RtlEnterCriticalSection
N/A	000C6D5D	0000	RtlLeaveCriticalSection

Figure 4.36. Functions called from kernel32.dll to ntdll.dll

After dumping and fixing IAT, reversing engineers can totally base on the dumped file to analyse the behavior of the file. This is the graph comparison:

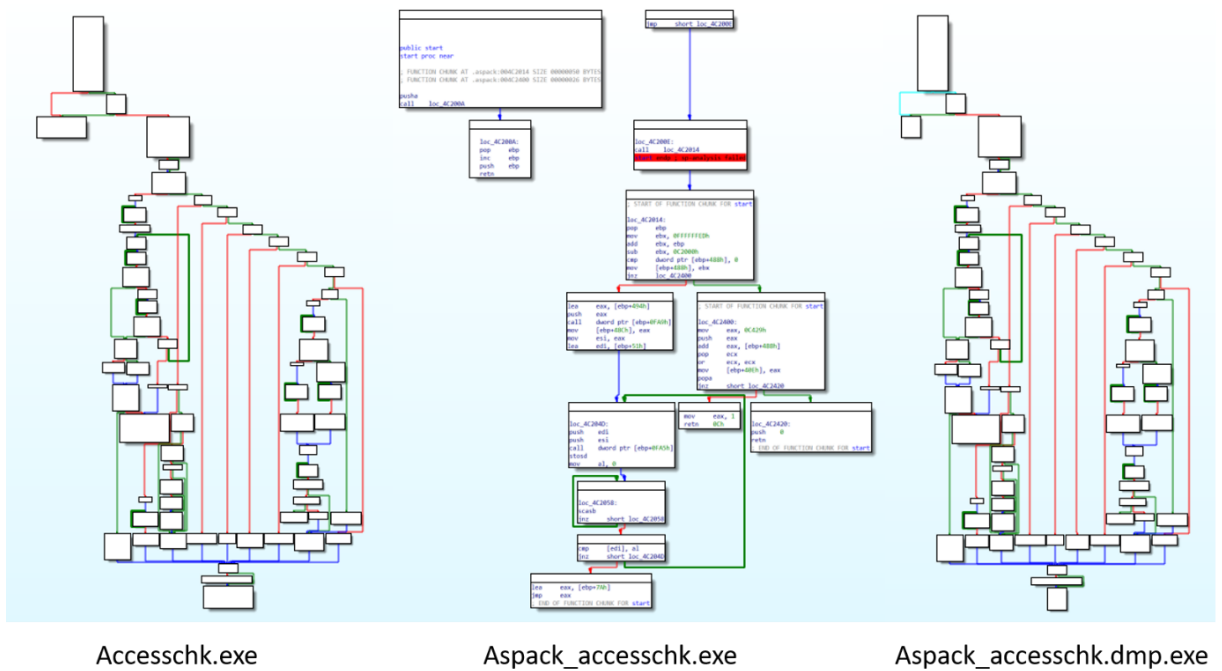


Figure 4.37. Graph comparison

4.3. Conclusion

In this chapter, we have detailed the implementation and experimental phases of the project. The implementation section covered various critical modules, including the data preparation module, which sets the foundation for the overall system, and the template setup module, which configures the templates based on graph similarities and Weisfeiler-Lehman vectors. Additionally, we implemented the packer identification and OEP detection module to recognize packer types and define the OEP, as well as the IAT rebuild module to ensure accurate resolution of the import address table for dumped files. Finally, we presented the diagram of the solution to visualize the complete workflow of the whole system.

The experimental section validated these implementations by conducting tests on real-world packed binaries. We demonstrated the tool set's ability to accurately detect packers and extract original payloads, both manually and with our automated PE Dumping module. The experiments revealed the effectiveness of the CFG generation and packer identification modules, which significantly contributed to the success of recognizing and unpacking files. We achieved high

accuracy in IAT reconstruction and payload extraction, as illustrated by the experimental results.

Overall, the chapter concludes that the implemented system is capable of efficiently analyzing and unpacking packed executables, making it a powerful tool for reverse engineering and malware analysis. The results demonstrate the practicality of combining symbolic execution, graph analysis, and automated unpacking techniques to handle modern packer obfuscation challenges.

CONCLUSION

Achievements

In summary, this research has led to several key accomplishments and contributions to the field of malware analysis, particularly in the context of dynamic symbolic execution and PE file unpacking. The results achieved are as following key points:

Understanding dynamic symbolic execution and its applications: A deep understanding of dynamic symbolic execution was developed throughout this work. We explored how DSE-based tools, such as BE-PUM and angr, operate, including their methodologies for analyzing binary code through symbolic execution. This exploration included how these tools traverse multiple execution paths in parallel, providing a comprehensive examination of the underlying code. In addition, we explored how DSE can be utilized for tasks such as vulnerability detection, malware analysis, and unpacking packed executables.

Utilization of be-pum for packer identification and oep detection: The research capitalized on BE-PUM's ability to generate logs and control flow graphs to assist in identifying various packers. Specifically, we worked with 12 different packers and leveraged graph similarity techniques to detect the OEP in packed Windows x86 Portable Executable files. By comparing the CFGs of packed and unpacked binaries, we were able to effectively detect the transition from the unpacking routine to the original executable code, which is critical for successful unpacking process.

Template building for precise detection: To improve detection accuracy, templates were rebuilt for each of the packers under analysis. These templates included key characteristics of each packer's unpacking routine, allowing us to refine the detection process and increase the precision of OEP identification. This template-based approach ensures that our detection system becomes more robust and adaptable to the specific unpacking patterns of different packers.

Development of CLI applications: As part of the practical contributions of this research, two command-line interface applications were built. The first CLI tool is designed to detect the type of packer used and identify the OEP of a packed file. This tool provides an automated mechanism for analyzing packed binaries and pinpointing the critical transition point between the packed and unpacked code. The second CLI tool focuses on dumping the original payload of the unpacked binary and reconstructing the IAT, which is essential for restoring the executable's functionality post-unpacking. These tools represent significant progress toward the automation of unpacking and analysis.

Limits

Although my research has aforementioned achievements, it still has limitations which need considering:

- The primary target of this research has been packed x86 PE files, which may be somewhat outdated in modern malware contexts. Many contemporary threats utilize 64-bit architectures, and the current work does not cover x64 binaries. Additionally, we have only addressed single-layer packed files, which simplifies the unpacking process compared to multi-layer packed files. Handling multi-layer packing will require further research and development;
- The current system relies heavily on BE-PUM for generating the logs and CFGs that are essential for packer identification and OEP detection. This dependency limits the flexibility and autonomy of the system, as it cannot function independently of BE-PUM. Moreover, the application developed is not yet an end-to-end solution, meaning that a fully integrated, user-friendly system is still lacking;
- While the CLI tool for dumping and IAT reconstruction represents a significant advancement, it still requires further refinement. The process of dumping PE files and accurately rebuilding their IATs is complex, and there are still some cases where the results are not as precise as

desired. Further research is needed to optimize these functions and ensure that they consistently yield correct and executable results, especially when dealing with more challenging packers.

Future Works

Moving forward, there are several directions that this research could take to overcome the existing limitations and further enhance the capabilities of the tools developed:

- A key area for future research is the development of a more efficient algorithm for extracting the executable payload from packed files. This could involve optimizing the current techniques used for OEP detection and IAT reconstruction, as well as incorporating new methods that improve the accuracy and efficiency of the extraction process;
- Another important direction for future work is to broaden the scope of the system to handle unknown packers and complex packers like Themida, VMProtect, and other advanced obfuscation methods. These packers use sophisticated techniques to hide their payloads, and developing strategies to unpack such files will require significant research, particularly in the areas of devirtualization and anti-debugging, anti-tampering techniques;
- As modern malware continues to evolve, many malware authors are adopting multi-layer packing techniques, where an executable is packed multiple times with different packers. Developing methods to unpack multiple layers is an important future goal. This will likely involve iterating the unpacking process and detecting multiple OEPs to eventually reach the original executable;
- There is a clear need to move towards a fully integrated, end-to-end application that combines the functionality of BE-PUM with the custom CLI tools developed during this research. Such an application would provide a complete solution for analyzing packed executables, from

initial detection through to payload extraction and IAT reconstruction, all within a single user-friendly interface;

- Finally, expanding the research to support x64 binaries and other file formats (such as ELF files used in Unix-like systems) will be necessary to maintain relevance in the evolving field of malware analysis. Future work should explore how the current techniques can be adapted to work with 64-bit systems and non-Windows file formats to increase the versatility and applicability of the system.

BIBLIOGRAPHY

- [1] Weisfeiler-Lehman graph kernels. Available at: <https://www.jmlr.org/papers/volume12/shervashidze11a/shervashidze11a.pdf> (Accessed: 26 March 2024).
- [2] Cheng, B. et al. (2021) {obfuscation-resilient} executable payload extraction from packed malware, USENIX. Available at: <https://www.usenix.org/conference/usenixsecurity21/presentation/cheng-binlin> (Accessed: 27 March 2024).
- [3] Hai, N.M., Ogawa, M. and Tho, Q.T. (2017) ‘Packer identification based on metadata signature’, Proceedings of the 7th Software Security, Protection, and Reverse Engineering / Software Security and Protection Workshop [Preprint]. doi:10.1145/3151137.3160687.
- [4] Hungpthan, Hungpthan/OEP-detection-based-on-graph-similarity: Original entry point detection based on graph similarity, GitHub. Available at: <https://github.com/hungpthan/oep-detection-based-on-graph-similarity> (Accessed: 27 March 2024).
- [5] Isawa, R., Kamizono, M. and Inoue, D. (2013) ‘Generic unpacking method based on detecting original entry point’, Neural Information Processing, pp. 593–600. doi:10.1007/978-3-642-42054-2_74.
- [6] Ollydbg_tut26, Oday in {REA_TEAM}. Available at: https://kienmanowar.wordpress.com/category/ollydbg-tutorials/ollydbg_tut26/ (Accessed: 27 March 2024).
- [7] Packing-Box, Packing-box/dataset-packed-PE: Dataset of packed PE samples, GitHub. Available at: <https://github.com/packing-box/dataset-packed-pe> (Accessed: 27 March 2024).
- [8] PSUCyberSecurityLab, PSUCyberSecurityLab/AIFORCYBERSECURITY, GitHub. Available at: <https://github.com/PSUCyberSecurityLab/AIforCybersecurity/tree/main/Chapter6-Malware-Classification> (Accessed: 27 March 2024).
- [9] Sikorski, M. (2012) Practical malware analysis. No Starch Press,us.
- [10] Vishnyakov, A. et al. (2020) ‘SYDR: Cutting edge dynamic symbolic execution’, 2020 Ivannikov Ispras Open Conference (ISPRAS) [Preprint]. doi:10.1109/ispras51486.2020.00014.
- [11] Weisfeiler Leman graph isomorphism test (2024) Wikipedia. Available at: https://en.wikipedia.org/wiki/Weisfeiler_Leman_graph_isomorphism_test (Accessed: 27 March 2024).
- [12] Welcome to ANGR’s documentation!#, angr documentation. Available at: <https://docs.angr.io/en/latest/> (Accessed: 27 March 2024).
- [13] X86 (2024) Wikipedia. Available at: <https://en.wikipedia.org/wiki/X86> (Accessed: 27 March 2024).